MDPI

Article

Role-Symmetric EAP-EDHOC with Hybrid Post-Quantum Cryptography for 5G Secondary Authentication and Application-Server Access

Adi Panca Saputra Iskandar [1,4], SeongHan Shin [2], Yongho Ko [1], I Wayan Adi Juliawan Pawana [1,4], Hoseok Kwon [1] and Ilsun You [1,3,*]

[1] Department of Cybersecurity, Kookmin University, Seoul 02707, Republic of Korea; adipancaiskandar@gmail.com (A.P.S.I.); koyh0911@kookmin.ac.kr (Y.K.); adijuliawan@kookmin.ac.kr (I.W.A.J.P.)
[2] Cyber Physical Security Research Center, National Institute of Advanced Industrial Science and Technology (AIST), Tokyo 135-0064, Japan; seonghan.shin@aist.go.jp
[3] KMU Global Research Center for ICT Convergence Security, Kookmin University, Seoul 02707, Republic of Korea
[4] Department of Electrical Engineering, Udayana University, Badung, Bali 80361, Indonesia
* Correspondence: isyou@kookmin.ac.kr

Abstract

5G secondary authentication lets an external data network (DN) authenticate user equipment (UE) through the Extensible Authentication Protocol (EAP), but widely used methods such as EAP-TLS exchange certificate chains over many EAP round-trips and rely on elliptic-curve cryptography that a quantum adversary can break. Moving to post-quantum key encapsulation also breaks the role symmetry of authenticated key exchange, because a key encapsulation mechanism (KEM) implicitly authenticates only the decapsulating party. We propose a role-symmetric hybrid post-quantum EAP-EDHOC (Ephemeral Diffie–Hellman Over COSE) protocol that restores this symmetry: the UE and the DN authentication server each receive exactly one X-Wing (X25519 + ML-KEM-768) encapsulation to a one-time static key, each authenticate with a detached ML-DSA-44 signature, and each rotate and erase their key after every run. The mirrored rotation gives both parties forward secrecy and post-compromise security, and the independent signatures give Key-Compromise Impersonation (KCI) resistance in both directions. The exported Extended Master Session Key bootstraps application-layer keys for login-free server access. We model nine security requirements in ProVerif for seven methods, including a negative control without key erasure. An implementation on Open5GS, UERANSIM, and FreeRADIUS, run through the full 5G core path, shows that the symmetry extends to the traffic: of the 12,227-byte handshake, the UE sends 6,114 bytes and the server 6,113 bytes. The cost, in contrast, is asymmetric and lies in transport fragmentation rather than in cryptography: the proposal computes slightly more than hybrid EAP-TLS but needs half its EAP round-trips.

Keywords: symmetric mutual authentication; asymmetric cryptography; hybrid post-quantum cryptography; EDHOC; 5G secondary authentication; X-Wing KEM; ML-DSA; formal verification

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Symmetry* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

A 5G operator serves many network slices and external data networks, such as enhanced mobile broadband (eMBB), ultra-reliable low-latency communication (URLLC),

massive machine-type communication (mMTC), enterprise and industrial networks, and Multi-access Edge Computing (MEC). Primary authentication through 5G Authentication and Key Agreement (5G-AKA) or EAP-AKA′ only proves that the subscriber is legitimate for the operator and does not grant access to these services [1,2]. It is also not a sufficient trust anchor on its own: formal analyses of 5G-AKA have revealed protocol-level weaknesses [3] and a privacy attack that survives the move from 4G to 5G [4]. 5G therefore defines *secondary authentication* based on the Extensible Authentication Protocol (EAP) [1,5], in which the user equipment (UE) authenticates to a data network (DN) Authentication, Authorization, and Accounting (DN-AAA) server owned by the service provider, so that connectivity is authenticated by the operator while service-level authorization stays with the service owner. Dedicated UE–DN authentication frameworks [6] and analyses of the EAP-TLS profile [1,7] show the growing attention paid to this step.

Figure 1 places the flow in the 3GPP architecture. After primary authentication between the Access and Mobility Management Function (AMF) and the Authentication Server Function (AUSF), and network slice selection through the Network Slice Selection Function (NSSF) and the Network Repository Function (NRF), the UE reaches the DN-AAA server over the Session Management Function (SMF)/User Plane Function (UPF) path to run secondary authentication; the DN-AAA then bootstraps keys toward an application server (AS), using the Extended Master Session Key (EMSK), so the UE reaches the AS without a further login.

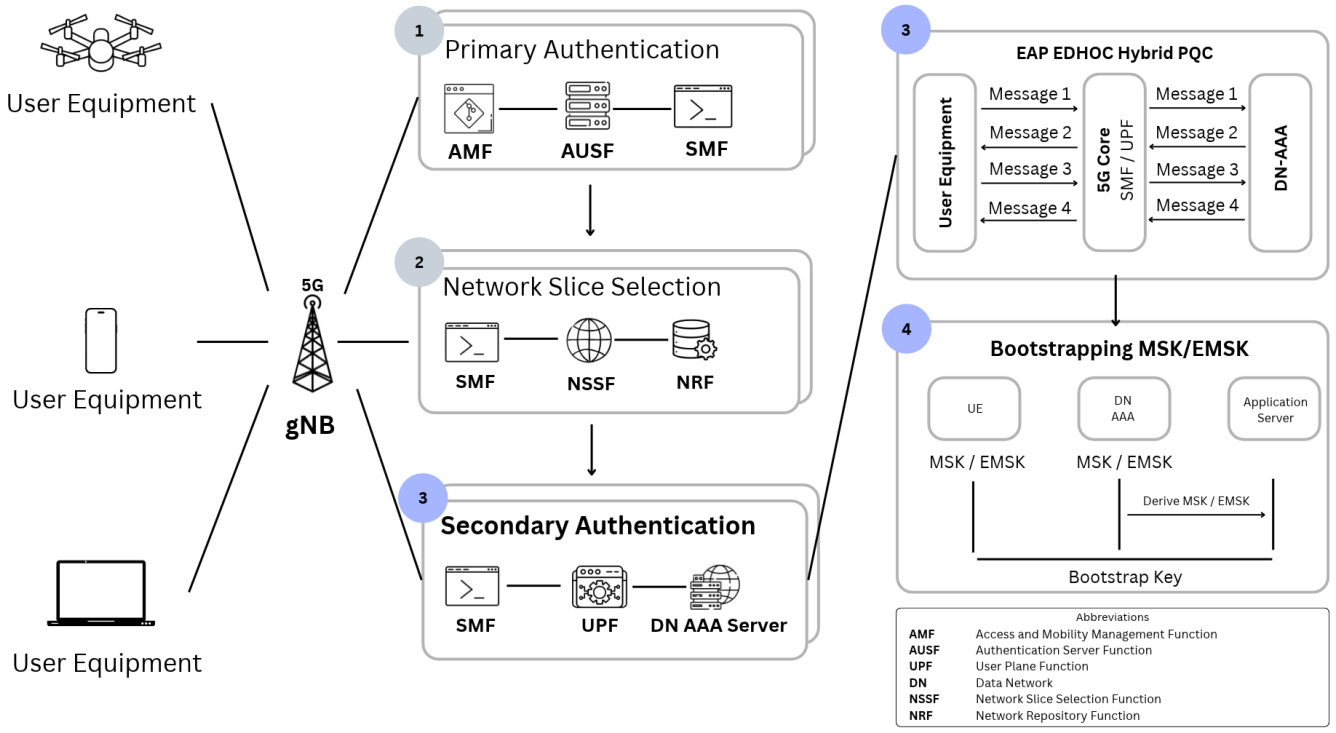


Figure 1. 5G ~~secondary~~ authentication ~~phase~~ phases and ~~Extended Master Session Key (~~MSK/EMSK ~~) key bootstrapping phase. AMF: Access and Mobility Management Function; AUSF: Authentication Server Function; SMF: Session Management Function; UPF: User Plane Function; DN-AAA: data network Authentication, Authorization, and Accounting server~~key bootstrapping.

EAP-TLS (Transport Layer Security) ~~[7] is costly for~~ [7,8], which today runs over TLS 1.3 [9], exchanges X.509 certificate chains inside the TLS handshake, which adds bytes and EAP round-trips on the links of constrained Internet of Things (IoT) and industrial devices~~because of certificate exchange and the TLS handshake~~. EDHOC (Ephemeral Diffie–

Hellman Over COSE) [10,11], a lightweight authenticated key exchange based on the Concise Binary Object Representation (CBOR) and CBOR Object Signing and Encryption (COSE), has been carried into EAP by the EAP-EDHOC draft [12] and is markedly more efficient than Datagram TLS (DTLS) 1.3 [13,14]. EDHOC, however, still relies on elliptic-curve Diffie–Hellman, which Shor's algorithm breaks [15] and which is exposed to *harvest-now, decrypt-later* attacks. Hybrid post-quantum cryptography (PQC) is the recommended transition path because it keeps interoperability with classical systems [16–20]; X-Wing [21, 22] is a compact hybrid key encapsulation mechanism (KEM) that combines X25519 with ML-KEM-768. Real hybrid-PQC deployments of EAP-EDHOC in an open-source 5G system, and the effect of PQC payloads on Remote Authentication Dial-In User Service (RADIUS) encapsulation [23,24], remain largely unexplored [25–28].

Symmetry perspective. Authenticated key exchange is governed by a symmetry between its two parties. Diffie–Hellman is inherently symmetric, since both sides contribute a share and derive the same secret, but a KEM is asymmetric: only the holder of the decapsulation key is implicitly authenticated. EDHOC's four methods also mix symmetric (SIG-SIG, STAT-STAT) and asymmetric, mirror-image (SIG-STAT, STAT-SIG) role assignments. Replacing Diffie–Hellman with a KEM therefore breaks the role symmetry unless the protocol restores it. Our design restores it explicitly: each party is the target of exactly one X-Wing encapsulation to a one-time static key, each party is authenticated by its own detached ML-DSA-44 signature, and each party rotates its static key after every run, which also replaces the ephemeral key pair as the source of forward secrecy. At the same time, we show that the protocol's *costs* are strongly asymmetric, cheap in computation and expensive in transport, which determines where deployment effort should go.

The contributions of this work are as follows:

1. A role-symmetric hybrid PQC EAP-EDHOC protocol for 5G secondary authentication, built on X-Wing with a ~~SIGMA-style~~ detached ML-DSA-44 signature gate that resists Key-Compromise Impersonation (KCI) and a symmetric ~~key rotation that provides~~ rotation of one-time static keys that provides both forward secrecy and post-compromise security without an ephemeral key pair.
2. A ProVerif [29,30] analysis of nine security requirements, comparing the proposal with the four classical EDHOC methods and two EAP-TLS baselines, with a negative-control model showing that forward secrecy rests on key erasure.
3. An end-to-end implementation on Open5GS [31], UERANSIM [32], and FreeRADIUS [33], with open-source code [34], including EMSK-based key bootstrapping of Object Security for Constrained RESTful Environments (OSCORE) for login-free application-server access.
4. A performance evaluation of computation, communication, EAP/RADIUS fragmentation, and packet-loss behaviour that quantifies the asymmetry between cryptographic and transport cost.

2. Related Work

Security analysis of classical EDHOC. RFC 9528 has been analysed symbolically by Jacomme et al. [35] (all four methods), Norrman et al. [36], and Kim et al. [37], and computationally by Günther and Mukendi [38], Cremers et al. [39], Cottier and Pointcheval [40], and Ferreira [41]; Pérez et al. [14] survey this work. A recurring finding is that the MAC-based methods (T1–T3) are vulnerable to KCI once the static Diffie–Hellman key leaks [38,39]. These works analyse EDHOC as specified, which remains purely classical.

Post-quantum and hybrid EDHOC. Fraile et al. [25] propose PQ-EDHOC, a fully post-quantum variant using NIST KEMs and signatures, evaluated with microbenchmarks on constrained hardware. A companion work [26] and IETF draft [27] add a signature-free,

KEM-based authentication method. Shin et al. [28] embed an ephemeral PQC KEM into EDHOC and omit the optional Message 4, and in follow-up work [42] they strengthen this hybrid EDHOC design toward post-quantum security against KCI. The present paper builds on that line of work and further hardens KCI resistance with a detached ML-DSA-44 gate that is independent of the KEM static keys. None of these proposals target 5G secondary authentication or integrate with EAP/RADIUS, and none combines KCI resistance with post-compromise key rotation.

5G secondary authentication. Gong et al. [6] propose new network functions for UE–DN mutual authentication, and the EAP-TLS profile of TS 33.501 Annex B has been formally analysed [1,7]. These works do not consider PQC or the EAP-over-RADIUS fragmentation that, as we show in Section 6, becomes the bottleneck once PQC payloads are introduced. No prior work combines hybrid PQC, a KCI-resistant gate independent of the KEM static key, formal verification, a live 5G implementation, and EMSK-based key bootstrapping.

Table 1 classifies the seven evaluated methods by the mechanism that authenticates each party. Read row by row, it exposes the role symmetry of each design: Method 0 (SIG-SIG) and Method 3 (STAT-STAT) are role-symmetric, while Methods 1 and 2 are mirror images of each other, each authenticating one party by signature and the other by a static-DH MAC. The static-DH MAC is the mechanism shown to be KCI-vulnerable [38,39]. EAP-TLS is role-symmetric, with entity authentication provided by the `CertificateVerify` signature; its HMAC `Finished` message is a universal key-confirmation step rather than a separate authentication mechanism. Only three methods are post-quantum, and only the proposed method pairs role symmetry with a signature that is both post-quantum and detached from the key-exchange static key. Although the proposal and EAP-TLS Hybrid PQC share the same primitives, they differ in how authentication is framed, which Section 6 shows matters more for latency than the primitives themselves.

Table 1. ~~Cryptographic building blocks~~ Authentication mechanisms of the seven evaluated methods~~: the signature- or message authentication code (MAC)-based mechanism authenticating the initiator and the responder. SIG: signature; STAT: static Diffie–Hellman key; DH: Diffie–Hellman; HKDF: HMAC-based key derivation function; PQC: post-quantum cryptography~~.

Method	Initiator authentication	Responder authentication	PQC
EAP-EDHOC Method 0 (SIG-SIG) [10,12]	Signature (Ed25519)	Signature (Ed25519)	No
EAP-EDHOC Method 1 (SIG-STAT) [10,12]	Signature (Ed25519)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Method 2 (STAT-SIG) [10,12]	Static-DH MAC (HKDF/HMAC-SHA256)	Signature (Ed25519)	No
EAP-EDHOC Method 3 (STAT-STAT) [10,12]	Static-DH MAC (HKDF/HMAC-SHA256)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Hybrid PQC (Section 4, over [12])	**Signature (ML-DSA-44, detached SIGMA gate)**	**Signature (ML-DSA-44, detached SIGMA gate)**	**Yes (hybrid)**
EAP-TLS Classic ~~[7]~~[8,9]	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	No
EAP-TLS Hybrid PQC ~~[7,51]~~ [8,43,44]	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Yes (hybrid)

SIG: signature; STAT: static Diffie–Hellman key; MAC: message authentication code; HKDF: HMAC-based key derivation function.

3. Preliminaries

3.1. 5G Secondary Authentication

EAP [5] defines three roles: the *peer* (the UE), the *authenticator*, which relays messages without understanding the method (the SMF), and the *EAP server*, which runs the method and decides (the DN-AAA) [1,2]. EAP frames travel over the SMF–UPF–DN-AAA path

of Figure 1 and are carried in RADIUS between the SMF and the DN-AAA [23,24]. Unlike primary authentication, which the AUSF terminates inside the operator's domain, secondary authentication is terminated by a DN-AAA that the operator does not control and runs only after the PDU session is established [1]. A successful method exports the Master Session Key (MSK) and the Extended MSK (EMSK) [5,45]. The MSK protects the UE–DN session; the EMSK never leaves the peer and the server and serves as a root for cryptographically separated keys through RFC 5295's Usage-Specific Root Keys (USRK). We use it to bootstrap an OSCORE [46] context with an application server without any further login.

3.2. X-Wing Hybrid KEM

X-Wing [21,22] combines X25519 with ML-KEM-768 (FIPS 203) [47,48], following the IETF hybrid terminology [19] and the rationale of hybrid KEMs for authenticated key exchange [20,49]. The combined secret stays secure as long as *at least one* component is unbroken. Algorithm 1 summarises its three operations. The public key $pk = ek_{\mathrm{PQ}} \,\|\, pk_X$ is $1184 + 32$ bytes. Encapsulation runs ML-KEM-768 and an ephemeral X25519 exchange and merges both shares with a single SHA3-256 *combiner* that also binds both public shares, which blocks re-encapsulation and mismatch attacks [39,50]; decapsulation recomputes the same 32-byte secret. We follow the X-Wing specification [22] exactly: the key pair is derived from a 32-byte seed with SHAKE256, and the combiner hashes $ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X$ followed by the 6-byte label `5c2e2f2f5e5c`. Our implementation reproduces all test vectors of the specification for key generation, encapsulation, and decapsulation. As the specification permits for a key that never leaves its device, each party stores the expanded secret key $dk_{\mathrm{PQ}} \,\|\, sk_X$ and takes pk_X from its own public key during decapsulation instead of recomputing it.

X-Wing is IND-CCA secure under standard assumptions and, by exploiting ML-KEM's ciphertext binding, needs weaker assumptions than generic combiners [21]. Unlike the negotiated combiners of hybrid TLS 1.3 ~~[51]~~[43,51], IKEv2 [52], or composite ML-KEM [20,53], it is a fixed instantiation without runtime negotiation, in line with deployed hybrids such as PQXDH [54,55] and post-quantum Noise [50], and compact enough for COSE/CBOR. Note that a KEM is asymmetric by construction: the encapsulator learns nothing about the decapsulator's long-term key, and only the decapsulator is implicitly authenticated. Section 4.5 shows how the protocol compensates for this.

Algorithm 1 X-Wing hybrid KEM: KeyGen(), Encaps(pk), and Decaps(ct, sk)

KeyGen():
  ~~$(dk_{\mathrm{PQ}}, ek_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.KeyGen()~~ $seed \leftarrow \mathsf{random}(32)$; ~~$(sk_X, pk_X) \leftarrow$ X25519.KeyGen()~~
  $(d, z, sk_X) \leftarrow \mathsf{SHAKE256}(seed, 96)$
  $(dk_{\mathrm{PQ}}, ek_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.KeyGen_internal(d, z); $pk_X \leftarrow \mathsf{X25519}(sk_X, \mathsf{base})$
  return $pk \leftarrow ek_{\mathrm{PQ}} \,\|\, pk_X$ $(1184 + 32 = 1216$ bytes$)$, $sk \leftarrow dk_{\mathrm{PQ}} \,\|\, sk_X$
Encaps($pk = ek_{\mathrm{PQ}} \,\|\, pk_X$):
  $(ss_{\mathrm{PQ}}, ct_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.Encaps(ek_{PQ}); $(sk_e, pk_e) \leftarrow \mathsf{X25519.KeyGen}()$
  $ss_X \leftarrow \mathsf{X25519}(sk_e, pk_X)$
  $ss \leftarrow \mathsf{SHA3\text{-}256}(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \mathtt{label}_{\text{X-Wing}})$ ▷ combiner
  return $(ss,\ ct \leftarrow ct_{\mathrm{PQ}} \,\|\, pk_e)$
Decaps($ct = ct_{\mathrm{PQ}} \,\|\, pk_e, sk = dk_{\mathrm{PQ}} \,\|\, sk_X$):
  $ss_{\mathrm{PQ}} \leftarrow$ ML-KEM-768.Decaps$(ct_{\mathrm{PQ}}, dk_{\mathrm{PQ}})$; $ss_X \leftarrow \mathsf{X25519}(sk_X, pk_e)$
  return $ss \leftarrow \mathsf{SHA3\text{-}256}(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \mathtt{label}_{\text{X-Wing}})$

4. Proposed EAP-EDHOC Hybrid PQC Protocol

4.1. Assumptions and Design Rationale

In a provisioning phase (Algorithm 2), the UE (Initiator) and the DN-AAA (Responder) each generate a long-term X-Wing key pair and a long-term ML-DSA-44 signing key, and

each holds the peer's long-term X-Wing public key. The peer's verification key (VK_R, VK_I) is conveyed in `Plaintext2`/`Plaintext3` and bound to the transcript by the corresponding signature. EAP-EDHOC messages are carried over EAP [5] and RADIUS [23,24] between the pass-through authenticator (SMF/UPF) and the DN-AAA, following TS 33.501 [1]. The Dolev–Yao adversary [56] controls the network.

The ~~protocol follows the SIGMA (SIGn-and-MAc)design [57]:~~ X-Wing static keys are *one-time* keys. Each party generates its next key pair at the start of its part of the run and announces the public key early, the UE in EAD_1 of `Message1` and the DN-AAA in EAD_2 of `Message2`; both announcements are bound to the transcript and covered by the signatures. Each party switches to its new key pair and erases the old secret key once the run completes, so every static key pair serves exactly one session. Because both next public keys are known before `Message3`, the loss of a later message never leaves a party without its peer's next key, which keeps recovery simple (Section 4.4). The DN-AAA keeps one record per UE, holding (sk_B, pk_B) for that UE together with (pk_A, VK_I), and selects it from the EAP identity of the session, which reaches it in the RADIUS Access-Request before `Message1`; no extra field is added to the EDHOC messages. Each party keeps its keys in a preallocated two-slot file, and every write issues a single `fdatasync`.

Authentication follows the sign-and-MAC principle of SIGMA [57], although the message flow is not that of SIGMA-I: the Initiator's identity travels in `Message1`, protected by the Responder's one-time KEM key, rather than after the Responder has authenticated. X-Wing provides confidentiality and key agreement, while detached ML-DSA-44 signatures [58] are the sole source of entity authentication. Because the signing keys are independent of the X-Wing static keys, leaking a KEM static key does not enable impersonation; this closes the KCI gap of a MAC-only Method 4 variant [38,39]. We choose ML-DSA-44 (NIST category 2) rather than ML-DSA-65 to limit fragmentation: its 1312-byte key and 2420-byte signature are markedly smaller, and its margin still exceeds that of the Ed25519 signatures of T0–T3. Being nominally below the category-3 strength of ML-KEM-768, this is a deliberate size/security trade-off.

Algorithm 2 Long-term key generation (provisioning)

1: **At UE:** $(sk_A, pk_A) \leftarrow \mathsf{X\text{-}WING.KeyGen}()$; $(sk_{SIG,I}, VK_I) \leftarrow \mathsf{ML\text{-}DSA\text{-}44.KeyGen}()$
2: **At DN-AAA:** $(sk_B, pk_B) \leftarrow \mathsf{X\text{-}WING.KeyGen}()$; $(sk_{SIG,R}, VK_R) \leftarrow \mathsf{ML\text{-}DSA\text{-}44.KeyGen}()$
3: **return** $(pk_A, sk_A, sk_{SIG,I}, VK_I)$, $(pk_B, sk_B, sk_{SIG,R}, VK_R)$

4.2. Handshake

Figure 2 shows the exchange inside the 3GPP secondary-authentication flow, with the SMF as EAP authenticator and the DN-AAA as EAP server reached over RADIUS.

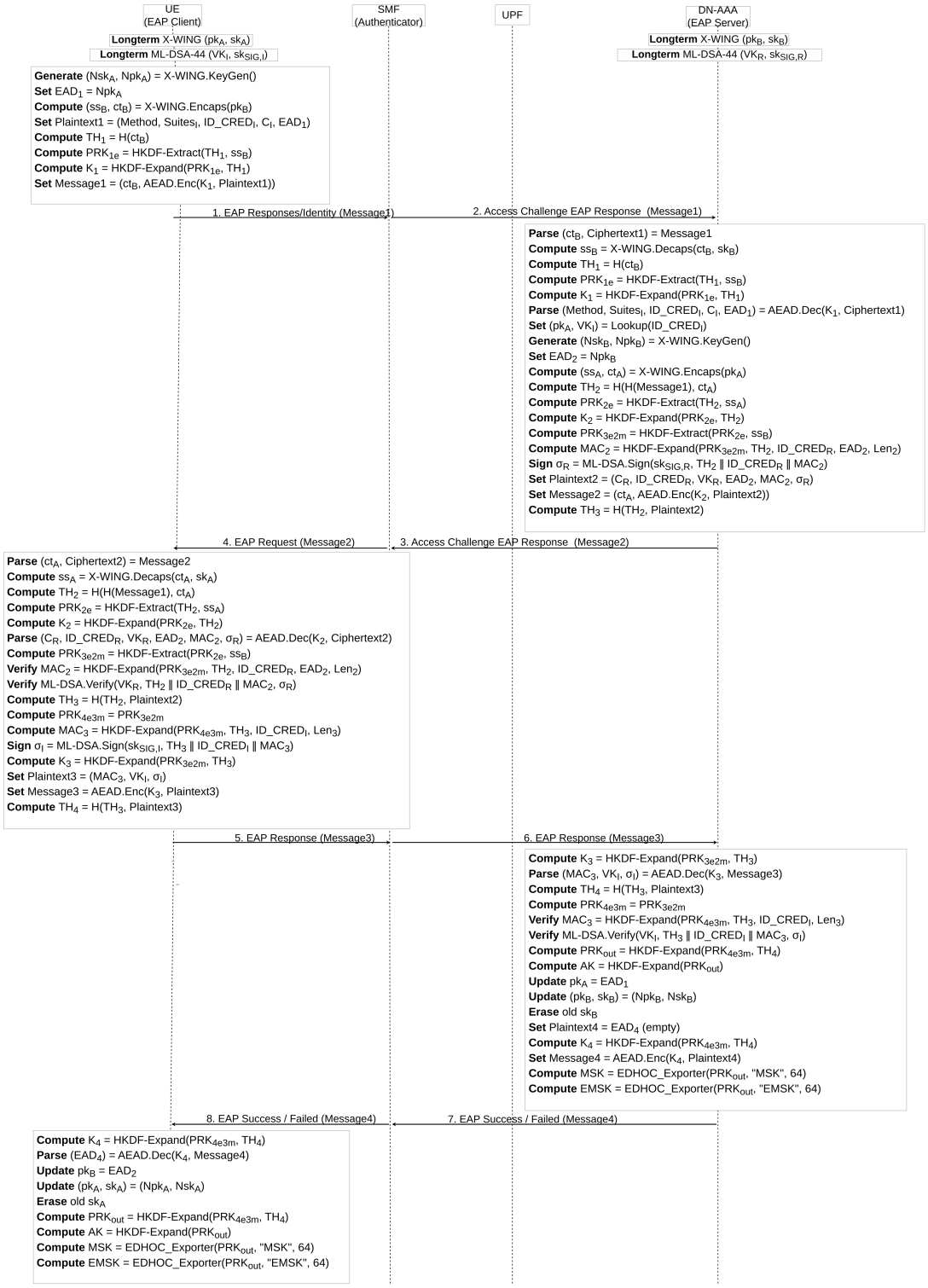


Figure 2. EAP-EDHOC hybrid PQC protocol~~for 5G secondary authentication~~.

Message 1 (Algorithm 3). The UE generates its next one-time key pair, encapsulates a fresh secret to the DN-AAA's ~~long-term~~ current one-time key pk_B, and encrypts `Plaintext1`, which announces Npk_A in EAD_1, under a transcript-derived key K_1. No ephemeral key pair is generated: forward secrecy comes from the erasure of the one-time static keys (Section 4.5). `Plaintext1` carries ID_CRED_I so that the DN-AAA can retrieve pk_A before building `Message2`. Because K_1 depends ~~on pk_B~~only on ss_B, ID_CRED_I ~~becomes retroactively recoverable if~~ is exposed if the *current* sk_B is ~~later compromised ;~~ compromised before the run; once the run completes and sk_B is erased, a later compromise

no longer reveals it, which Section 5 ~~confirms this formallyand Section 7 discusses how per-session key rotation bounds it~~checks formally.

Algorithm 3 Construction of `Message1` (Initiator)

Require: ~~pk_B (long-term public key of DN-AAA)~~ pk_B (current one-time public key of the DN-AAA)
Ensure: Message1
1: ~~$(sk_X, pk_X) \leftarrow$ X-WING.KeyGen()~~ $(Nsk_A, Npk_A) \leftarrow$ X-WING.KeyGen(); $EAD_1 \leftarrow Npk_A$ ▷ next key of UE
2: $(ss_B, ct_B) \leftarrow$ X-WING.Encaps(pk_B)
3: Plaintext1 $\leftarrow$ (Method, Suites$_I$, ID_CRED_I, C_I, EAD_1)
4: ~~$TH_1 \leftarrow H(pk_X, ct_B)$~~ $TH_1 \leftarrow H(ct_B)$
5: $PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_B)
6: $K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1)
7: ~~Message1 $\leftarrow (ct_B, pk_X,$ AEAD.Enc$(K_1,$ Plaintext1))~~ Message1 $\leftarrow (ct_B,$ AEAD.Enc$(K_1,$ Plaintext1))
8: **return** Message1

Message 2 (Algorithm 4). ~~The~~ Using the UE record selected from the EAP identity, the DN-AAA decapsulates, retrieves (pk_A, VK_I) from ID_CRED_I, and encapsulates to the UE's ~~ephemeral key and to its long-term key~~ one-time key pk_A. The ciphertext ct_A travels in the clear part of `Message2`, and ss_A keys the encryption of `Plaintext2`, so the Responder's identity is readable only by the holder of sk_A. It also generates its next key pair and announces Npk_B in EAD_2, which MAC_2 covers. The DN-AAA signs $TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2$ with a key independent of sk_B, giving KCI-resistant Responder authentication.

Algorithm 4 EDHOC `Message2` (DN-AAA $\rightarrow$ UE)~~with ML-DSA-44 authentication~~

Require: ~~Message1, sk_B, pk_A, $sk_{SIG,R}$, VK_R~~ Message1, per-UE records, $sk_{SIG,R}$, VK_R
Ensure: Message2
1: ~~$(ct_B, pk_X,$ Ciphertext1$) \leftarrow$ Message1~~ $(ct_B,$ Ciphertext1$) \leftarrow$ Message1
2: $ss_B \leftarrow$ X-WING.Decaps(ct_B, sk_B)
3: ~~$TH_1 \leftarrow H(pk_X, ct_B)$~~ $TH_1 \leftarrow H(ct_B)$
4: $PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_B)
5: $K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1)
6: (Method, Suites$_I$, ID_CRED_I, C_I, EAD_1) $\leftarrow$ AEAD.Dec$(K_1,$ Ciphertext1)
7: ~~$(ss_e, ct_e) \leftarrow$ X-WING.Encaps(pk_X)~~ $(pk_A, VK_I) \leftarrow$ Lookup(ID_CRED_I)
8: ~~$TH_2 \leftarrow H(H($Message1$), ct_e)$~~ $(Nsk_B, Npk_B) \leftarrow$ X-WING.KeyGen(); $EAD_2 \leftarrow Npk_B$ ▷ next key of DN-AAA
9: ~~$PRK_{2e} \leftarrow$ HKDF-Extract(TH_2, ss_e)~~ $(ss_A, ct_A) \leftarrow$ X-WING.Encaps(pk_A)
10: ~~$K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)~~ $TH_2 \leftarrow H(H($Message1$), ct_A)$
11: ~~$(ss_A, ct_A) \leftarrow$ X-WING.Encaps(pk_A)~~ $PRK_{2e} \leftarrow$ HKDF-Extract(TH_2, ss_A)
12: $K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)
13: $PRK_{3e2m} \leftarrow$ HKDF-Extract(PRK_{2e}, ss_B)
14: $MAC_2 \leftarrow$ HKDF-Expand$(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2)$
15: $\sigma_R \leftarrow$ ML-DSA.Sign$(sk_{SIG,R}, TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2)$ ▷ **independent of** sk_B
16: ~~Plaintext2 $\leftarrow (ct_A, C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$~~ Plaintext2 $\leftarrow (C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$
17: ~~Message2 $\leftarrow (ct_e,$ AEAD.Enc$(K_2,$ Plaintext2$))$~~ Message2 $\leftarrow (ct_A,$ AEAD.Enc$(K_2,$ Plaintext2$))$
18: $TH_3 \leftarrow H(TH_2,$ Plaintext2$)$
19: **return** Message2

Message 3 (Algorithm 5). The UE verifies MAC_2 and σ_R; this signature check is an authentication *gate* that must pass before the handshake continues. ~~The UE~~ Before sending `Message3`, the UE stores its pending keys (Section 4.4); it then authenticates itself in the mirror-image manner~~and announces a fresh X-Wing public key Npk_A in EAD_3~~. `Message3` carries no EAD.

Algorithm 5 EDHOC `Message3` (UE → DN-AAA), ~~KCI-resistant via the ML-DSA-44 gate~~

Require: ~~Message2, sk_X, sk_A, ss_B, $sk_{SIG,I}$, VK_I~~ Message2, sk_A, ss_B, $sk_{SIG,I}$, VK_I
Ensure: Message3
1: ~~$(ct_e$, Ciphertext2) ← Message2~~ $(ct_A$, Ciphertext2) ← Message2
2: ~~ss_e ← X-WING.Decaps(ct_e, sk_X)~~ ss_A ← X-WING.Decaps(ct_A, sk_A)
3: ~~$TH_2 \leftarrow H(H($Message1$), ct_e)$~~ $TH_2 \leftarrow H(H($Message1$), ct_A)$
4: ~~PRK_{2e} ← HKDF-Extract(TH_2, ss_e)~~ PRK_{2e} ← HKDF-Extract(TH_2, ss_A)
5: $K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)
6: ~~$(ct_A, C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$ ← AEAD.Dec$(K_2$, Ciphertext2)~~
$(C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R) \leftarrow$ AEAD.Dec$(K_2$, Ciphertext2)
7: $PRK_{3e2m} \leftarrow$ HKDF-Extract(PRK_{2e}, ss_B)
8: **Verify** $MAC_2 \stackrel{?}{=}$ HKDF-Expand$(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2)$ ▷ key confirmation
9: **Verify** ML-DSA.Verify$(VK_R,\ TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2,\ \sigma_R) \stackrel{?}{=}$ `valid` ▷ **auth gate**
10: **if either check fails: abort**
11: $TH_3 \leftarrow H(TH_2$, Plaintext2$)$
12: ~~ss_A ← X-WING.Decaps(ct_A, sk_A) PRK_{4e3m} ← HKDF-Extract(PRK_{3e2m}, ss_A)~~ $PRK_{4e3m} \leftarrow PRK_{3e2m}$ ▷ ss_A already entered at PRK_{2e}
13: ~~(Nsk_A, Npk_A) ← X-WING.KeyGen$()$; $EAD_3 \leftarrow Npk_A$~~ **store** NEXT $\leftarrow (Nsk_A, Npk_A, Npk_B = EAD_2)$ ▷ write-ahead, one `fdatasync`
14: ~~MAC_3 ← HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, EAD_3, Len_3)$~~ $MAC_3 \leftarrow$ HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, L$
15: $\sigma_I \leftarrow$ ML-DSA.Sign$(sk_{SIG,I},\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3)$ ▷ **independent of** sk_A
16: $K_3 \leftarrow$ HKDF-Expand(PRK_{3e2m}, TH_3)
17: ~~Plaintext3 ← $(EAD_3, MAC_3, VK_I, \sigma_I)$~~ Plaintext3 ← (MAC_3, VK_I, σ_I)
18: Message3 ← AEAD.Enc$(K_3$, Plaintext3$)$
19: $TH_4 \leftarrow H(TH_3$, Plaintext3$)$
20: **return** Message3

Message 4 and key export (Algorithms 6 and 7). The DN-AAA verifies the Initiator's gate, finalises the key schedule, ~~rotates its own keypair~~commits its new record, erases the old secret key, and returns ~~a fresh Npk_B in~~ Message4 with an empty EAD_4 as the protected success indication. The UE switches to its new keys only after it has verified `Message4`. The UE processes `Message4`, and both endpoints export identical MSK and EMSK with the EDHOC exporter. The MSK is returned to the 5G core; the EMSK is retained for key bootstrapping.

Algorithm 6 EDHOC `Message4` (DN-AAA → UE), ~~KCI-resistant via the ML-DSA-44 gate~~

Require: ~~Message3, PRK_{3e2m}, TH_3, ss_A~~ Message3, PRK_{3e2m}, TH_3
Ensure: Message4
1: $K_3 \leftarrow$ HKDF-Expand(PRK_{3e2m}, TH_3)
2: ~~$(EAD_3, MAC_3, VK_I, \sigma_I)$ ← AEAD.Dec$(K_3$, Message3$)$~~ $(MAC_3, VK_I, \sigma_I) \leftarrow$ AEAD.Dec$(K_3$, Message3$)$
3: $TH_4 \leftarrow H(TH_3$, Plaintext3$)$
4: ~~PRK_{4e3m} ← HKDF-Extract(PRK_{3e2m}, ss_A)~~ $PRK_{4e3m} \leftarrow PRK_{3e2m}$
5: **Verify** ~~$MAC_3 \stackrel{?}{=}$ HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, EAD_3, Len_3)$~~
$MAC_3 \stackrel{?}{=}$ HKDF-Expand$(PRK_{4e3m}, TH_3, ID_CRED_I, Len_3)$ ▷ key confirmation
6: **Verify** ML-DSA.Verify$(VK_I,\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3,\ \sigma_I) \stackrel{?}{=}$ `valid` ▷ **auth gate**
7: **if either check fails: abort**
8: $PRK_{out} \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4); $AK \leftarrow$ HKDF-Expand(PRK_{out})
9: ~~(Nsk_B, Npk_B) ← X-WING.KeyGen$()$; $EAD_4 \leftarrow Npk_B$ $pk_A \leftarrow EAD_3$~~ $pk_A \leftarrow Npk_A$ (from EAD_1) ▷ update next key of UE
10: $(pk_B, sk_B) \leftarrow (Npk_B, Nsk_B)$~~Plaintext4 ← EAD_4~~; **store** the record (one `fdatasync`), then **erase** the old sk_B ▷ write-ahead
11: Plaintext4 ← $EAD_4 = \varnothing$ ▷ protected success
12: $K_4 \leftarrow$ HKDF-Expand(PRK_{4e3m}, TH_4)
13: Message4 ← AEAD.Enc$(K_4$, Plaintext4$)$
14: MSK ← `EDHOC_Exporter`$(PRK_{out}$, "MSK", 64$)$; EMSK ← `EDHOC_Exporter`$(PRK_{out}$, "EMSK", 64$)$
15: **return** Message4

Algorithm 7 `Message4` processing and session-key export (UE)

Require: Message4, PRK_{4e3m}, TH_4
Ensure: MSK, EMSK
1: $K_4 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4)$; ~~$(EAD_4) \leftarrow$ AEAD.Dec(K_4, Message4)~~ verify AEAD.Dec(K_4, Message4)
2: ~~$pk_B \leftarrow EAD_4$~~ $pk_B \leftarrow Npk_B$ (from EAD_2) ▷ update next key of DN-AAA
3: $(pk_A, sk_A) \leftarrow (Npk_A, Nsk_A)$; **erase** the old sk_A ▷ forward secrecy
4: $PRK_{\text{out}} \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4)$; $AK \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\text{out}})$
5: MSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\text{out}}, \text{“MSK”}, 64)$; EMSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\text{out}}, \text{“EMSK”}, 64)$
6: **return** (MSK, EMSK)

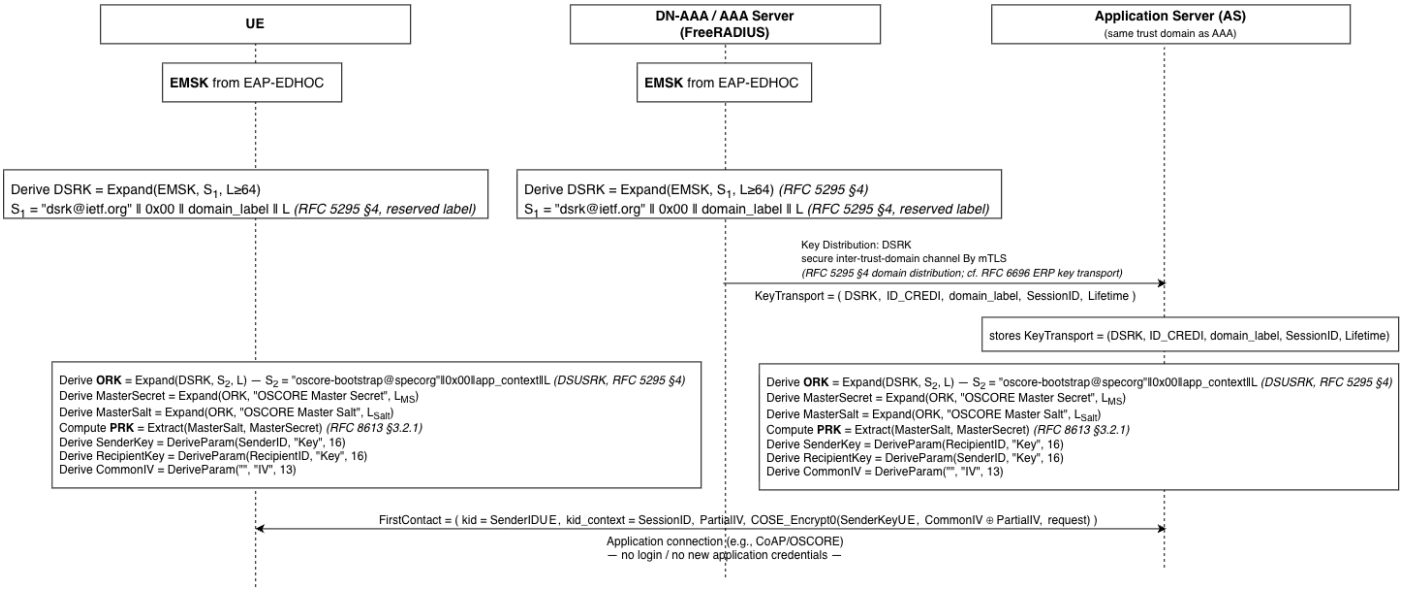


Figure 3. Application-server access ~~through key bootstrapping from the~~ via EMSK bootstrapping.

4.3. EMSK-Based Key Bootstrapping

Once secondary authentication succeeds, the UE and the DN-AAA share an identical EMSK. Figure 3 shows how it bootstraps keys that let the UE access the AS directly, and Figure 4 gives the key schedule, which combines RFC 5295's root-key hierarchy [45] with the OSCORE security context of RFC 8613 [46]. Every arrow is a single Expand or Extract call with a unique label, so compromise of one derived key does not disclose keys at another level (cryptographic separation, RFC 5295 Section 3). This phase uses only symmetric-key primitives, so its cost is negligible next to the asymmetric handshake that precedes it (Section 6.5).

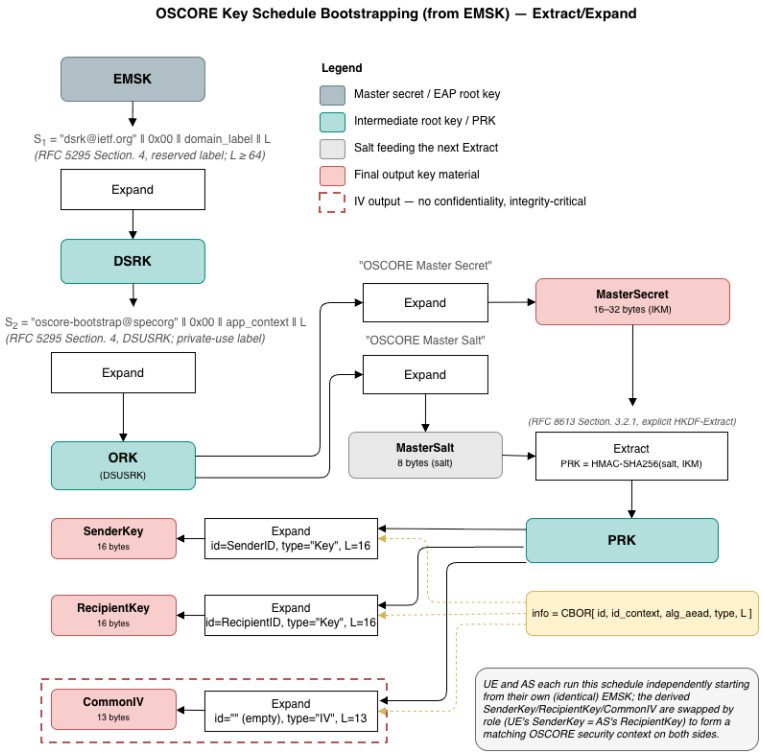


Figure 4. ~~Key schedule for bootstrapping keys from the~~ EMSK bootstrapping key schedule.

4.4. *Key Synchronization and Recovery*

Rotation changes state at both ends, so a lost message could leave the two parties on different key pairs. The protocol fixes the order of the durable writes so that at most one party keeps an old secret key, and only until the next successful run. The UE writes its pending state NEXT $= (Nsk_A, Npk_A, Npk_B)$ before it sends `Message3`. The DN-AAA verifies `Message3`, writes its new record $(Nsk_B, Npk_B, Npk_A, VK_I)$, and only then sends `Message4` (write-ahead); after sending it, the DN-AAA zeroes the slot that held the old sk_B, so it never keeps a grace key. `Message4` carries an empty EAD_4 and is the protected success indication that EAP-EDHOC mandates [12]; the UE promotes NEXT to its current state and erases the old sk_A only after it has verified `Message4`.

A UE that still holds NEXT starts its next run with it. If `Message4` or EAP-Success was lost, the DN-AAA has already committed, and the run succeeds with the new keys. If `Message3` was lost, the DN-AAA still holds its old record, cannot open `Plaintext1`, and answers with an EAP-Failure; the UE then retries once with its old keys. A timeout never triggers this fallback, and the UE keeps NEXT until a run completes: an EAP-Failure is not authenticated, so a forged failure can delay the UE but cannot make it discard a key, and no case requires re-provisioning.

Forward secrecy holds on every recovery path. The session key depends on both ss_A and ss_B, and the DN-AAA erases the old sk_B as soon as it accepts a run, so a later compromise of both parties cannot recover a session that the DN-AAA has accepted, even while the UE still holds its old sk_A; a dedicated ProVerif model checks exactly this case (Section 5). A run that the DN-AAA never accepted (`Message3` lost) produced no session key. A retained old sk_A can expose only the content of that run's `Message2`, a privacy rather than a key-secrecy loss.

4.5. Symmetry Analysis of the Design

We call a protocol *role-symmetric* if exchanging the Initiator and the Responder maps each party's cryptographic obligations onto those of the other. The proposed protocol satisfies this property in four respects, summarised in Table 2.

(i) Symmetric implicit authentication through two encapsulations. A single KEM authenticates only the decapsulator. The protocol therefore performs ~~two static encapsulationsin opposite directions,~~ exactly two encapsulations, one in each direction: ct_B to the DN-AAA ~~(~~in `Message1` ~~)~~ and ct_A to the UE ~~(~~in `Message2`~~), and feeds both secrets into~~ . Both secrets feed the key schedule (ss_A into PRK_{2e}, ss_B into PRK_{3e2m}~~,~~), and each message is encrypted under the secret it delivers to its receiver, K_1 from ss_B and K_2 from ss_A ~~into PRK_{4e3m})~~. This reproduces, with an asymmetric primitive, the two-sided contribution that static Diffie–Hellman gives naturally.

(ii) Symmetric explicit authentication. Each party signs its own transcript hash with a detached ML-DSA-44 key (σ_R over TH_2, σ_I over TH_3), and each verification is a mandatory gate. In contrast to the mirror-image methods T1/T2, neither party relies on a static-DH MAC, so KCI resistance holds for both directions rather than for one.

(iii) Symmetric key rotation. The UE announces Npk_{A} in ~~EAD_3~~ EAD_1 and the DN-AAA announces Npk_{B} in ~~EAD_4~~EAD_2; both replace their static key at the end of the run ~~. This~~ and erase the old secret key. Because every static key pair therefore serves a single session, this two-sided rotation ~~is what yields~~ yields forward secrecy (R5) as well as post-compromise security for both parties (R6), without an ephemeral key pair.

(iv) Symmetric key confirmation and output. MAC_2 and MAC_3 give mutual key confirmation, and both ends derive identical MSK/EMSK values.

The design keeps two intentional asymmetries. First, `Message1` must ~~reveal~~ carry ID_CRED_I so that the Responder can select pk_A; the Initiator's identity is therefore protected only under the Responder's ~~current long-term~~ *current* one-time key (R2). The outer EAP identity that selects the record is sent in the clear in our implementation; an anonymous or pseudonymous NAI, as recommended for EAP-EDHOC [12], would hide it and is left to deployment. Second, the four-message flow is not size-symmetric, as `Message2` and `Message3` carry the signatures and verification keys. ~~Section 6 shows, however, that the per-party computation and the~~ The total bytes sent in each direction are ~~nearly balanced~~nevertheless balanced: 6,114 B from the UE (`Message1`+`Message3`) and 6,113 B from the DN-AAA (`Message2`+`Message4`).

Table 2. Role symmetry of the proposed protocol~~: corresponding obligations of the Initiator (UE) and the Responder (DN-AAA)~~.

Obligation	**Initiator (UE)**	**Responder (DN-AAA)**
Static encapsulation received	ct_A to pk_A (in `Message2`)	ct_B to pk_B (in `Message1`)
Secret fed to key schedule	~~$ss_A \rightarrow PRK_{4e3m}$~~ $ss_A \rightarrow PRK_{2e}$	$ss_B \rightarrow PRK_{3e2m}$
Key-confirmation MAC sent	MAC_3	MAC_2
Detached signature (gate)	σ_I over TH_3, verified by DN-AAA	σ_R over TH_2, verified by UE
Static-key rotation and erasure	Npk_{A} in ~~EAD_3~~ EAD_1, old sk_A erased	Npk_{B} in ~~EAD_4~~ EAD_2, old sk_B erased
Session-key output	MSK, EMSK	MSK, EMSK
Intentional asymmetry	ID_CRED_I sent under the current pk_B-derived key	—

5. Formal Security Verification Using ProVerif

We analyse the protocol with ProVerif [29,30] in the Dolev–Yao model [56,59], following prior EDHOC analyses [35–37,41,60] and ISO/IEC 29128 [61]. The verification targets nine requirements:

- R1 **Mutual authentication**: each party is injectively authenticated to the other (agreement on the transcript).
- R2 **Identity concealment**: the persistent ~~identity~~ identities ID_CRED_I ~~is never~~ and ID_CRED_R are not learned by the adversary.
- R3 **Confidentiality**: application data and session keys remain secret.
- R4 **Secure key exchange**: MSK/EMSK are secret and agreed.
- R5 **Forward secrecy**: past session keys remain secret ~~after~~ when the long-term ~~keys leak~~state held after the session (signing keys and current static keys) leaks.
- R6 **Post-compromise security**: after a static key is compromised, security is restored once the rotated Npk_A/Npk_B is in use.
- R7 **AEAD protection** (Authenticated Encryption with Associated Data): plaintexts are integrity- and confidentiality-protected.
- R8 **Quantum resilience**: security holds against an adversary that can solve discrete logarithms, i.e., when X25519 and Ed25519 are broken (modelled by a quantum oracle).
- R9 **KCI resistance**: compromise of a party's static KEM key does not allow impersonation of the peer.

Table 3 maps each requirement to its ProVerif queries. We model seven variants: EDHOC T0 (SIG-SIG), T1 (SIG-STAT), T2 (STAT-SIG), T3 (STAT-STAT), the proposed protocol, and the two EAP-TLS baselines of Section 6, which are modelled as certificate-authenticated TLS 1.3 handshakes [9] in the EAP-TLS 1.3 profile [8] over the same query set. Each variant is analysed in three settings. In the *classical* setting, an active Dolev–Yao attacker controls the network and, in phase 1, learns all long-term keys of both parties. The *quantum* setting adds an oracle, available in every phase, that returns the discrete logarithm of any X25519 element and the Ed25519 signing key of any Ed25519 public key; this models Shor's algorithm against both an active attacker and recorded traffic, while no such oracle exists for ML-KEM-768 or ML-DSA-44. In the *KCI* setting, the long-term keys of one party are known to the attacker from the start, and we check that its peer is still authenticated to it. X-Wing is modelled ~~explicitly as ML-KEM-768 and~~ with its two halves kept separate: either secret half opens only its own component of a ciphertext, the shared secret needs both, and the quantum oracle recovers only the X25519 ~~joined by its combiner~~half, so the quantum setting checks the hybrid property instead of assuming it. Post-compromise security is checked in a dedicated model of the proposed protocol: after a run, the old static X-Wing keys of both parties are revealed, and a second run under the keys rotated through ~~EAD_3~~ EAD_1/~~EAD_4~~ EAD_2 is analysed. A property is fully secure (●) if it holds in both the classical and the quantum setting, and partial (◐) if it holds only in the classical setting. ~~All 23 models were verified with~~ For the proposed protocol, each run uses a fresh one-time static key pair per party, and the state leaked in phase 1 is the state a device actually holds after the run: the rotated X-Wing keys and both ML-DSA-44 signing keys. A negative-control model (`edhoc_m4v3_fs_noerase.pv`) additionally leaks the old, unerased keys of both parties; there the forward-secrecy and identity queries must fail, which shows that these properties rest on key erasure and that the models are not trivially secure. A recovery model (`edhoc_m4v3_recovery.pv`) covers the path of Section 4.4: the DN-AAA erases its old key at commit, whereas the UE keeps its old sk_A, as after a lost `Message4`, and both states leak in phase 1. All models target ProVerif 2.05; the models, run scripts, and outputs are publicly available [34].

Table 3. Security requirements and verification queries.

Security Requirement	Verification Query
Mutual authentication (R1)	Q1: `event(UE_send_M1(M1))` Q2: `inj-event(UE_recv_M2(...))` ⇒ `inj-event(DN_send_M2(...))` Q3: `inj-event(DN_recv_M3(...))` ⇒ `inj-event(UE_send_M3(...))` Q4: `inj-event(UE_recv_M4(...))` ⇒ `inj-event(DN_send_M4(...))`
Identity concealment (R2)	Q5: `attacker(ID_CREDI)`; Q5b: `attacker(ID_CREDR)`
Confidentiality (R3)	Q6: `attacker(app_data_UE)` Q7: `attacker(app_data_DN)`
Secure key exchange (R4)	Q8: `attacker(MSK_test)` Q9: `attacker(EMSK_test)`
Forward secrecy (R5)	Q8 && Q9 ~~, re-evaluated~~ after `phase 1` leaks ~~all long-term keys of both parties~~ the post-run state; no-erasure control must fail
Post-compromise security (R6)	Q10: `attacker(MSK_test2)` && Q11: `attacker(EMSK_test2)` after the old static keys are revealed
AEAD protection (R7)	Q2 && Q3 && Q4 (implicit via the `AEAD_dec` rewrite)
Quantum resilience (R8)	Q8 && Q9 in the quantum setting (X25519 discrete-log and Ed25519 key-recovery oracle)
KCI resistance (R9)	Q2 && Q4 with the UE's long-term keys leaked; Q3 with the DN's long-term keys leaked

Figure 5 shows the condensed ProVerif output. Q1–Q9 come from the classical model (`edhoc_m4v3_classical.pv`) and Q10–Q11 from the key-rotation model (`edhoc_m4v3_pcs.pv`). The reachability query (`is false`) is a sanity check that honest runs complete. The three injective correspondences (`is true`) confirm mutual authentication with transcript agreement for both directions of the handshake (R1), which formally reflects the role symmetry of Section 4.5. The `attacker_p1` queries confirm that, even after the static keys leak, the adversary learns neither the application data (R3) nor MSK/EMSK (R4, R5), and the key-rotation model confirms that a later session under the rotated keys regains secrecy (Q10, Q11; R6). Because ~~every run also derives its keys from a fresh ephemeral X-Wing encapsulation, the specific contribution of rotation is to confine the exposure of a compromised static key to the sessions that used it. The same leak does reveal~~ the protocol has no ephemeral key pair, rotation with erasure is also what provides forward secrecy: the state leaked after a run holds only the next one-time keys, which never protected that run. As the leaked state no longer contains the erased one-time keys, neither ID_CRED_I (Q5) nor ID_CRED_R (Q5b) is revealed; in the negative control, which keeps the old keys, Q5, ~~the intentional asymmetry discussed above~~Q5b, and Q6–Q9 fail, so forward secrecy and identity concealment rest on erasure. The negative control queries only secrecy: its extra keys leak in phase 1, after every authentication event, so its Q2–Q4 coincide with those of the classical model. Because authentication rests on signatures independent of the KEM static keys, leaking sk_A/sk_B breaks neither authentication nor KCI resistance (R9); replacing the static-DH MAC of a Method 4 variant [38,39] with a ~~SIGMA-style~~ detached signature is precisely what removes this exposure. In the recovery model, ID_CRED_I, the DN-AAA's application data, and MSK/EMSK stay secret although the UE still holds its old key, so forward secrecy does not depend on the UE erasing promptly.

```
Verification summary:
• Query  not event(UE_send_M1(M1)) is false.
• Query  inj-event(UE_recv_M2(MAC2,P2,TH2))
      ==> inj-event(DN_send_M2(MAC2,P2,TH2)) is true.
• Query  inj-event(DN_recv_M3(MAC3,M3,TH3))
      ==> inj-event(UE_send_M3(MAC3,M3,TH3)) is true.
• Query  inj-event(UE_recv_M4(M4,TH4))
      ==> inj-event(DN_send_M4(M4,TH4)) is true.
• Query  not attacker_p1(ID_CREDI[]) is false.
• Query  not event(UE_send_M1(WO,m)) is false.
• Query  inj-event(UE_recv_M2(WO,x,t,a,b))
      ==> inj-event(DN_send_M2(WO,x,t,a,b)) is true.
• Query  inj-event(DN_recv_M3(WO,x,t,a,b))
      ==> inj-event(UE_send_M3(WO,x,t,a,b)) is true.
• Query  inj-event(UE_recv_M4(WO,x,t,a,b))
      ==> inj-event(DN_send_M4(WO,x,t,a,b)) is true.
• Query  not attacker_p1(ID_CREDI[]) is true.
• Query  not attacker_p1(ID_CREDR[]) is true.
• Query  not attacker_p1(app_data_UE[]) is true.
• Query  not attacker_p1(app_data_DN[]) is true.
• Query  not attacker_p1(MSK_test[]) is true.
• Query  not attacker_p1(EMSK_test[]) is true.
• Query  not attacker(MSK_test2[]) is true.
• Query  not attacker(EMSK_test2[]) is true.
```

Figure 5. ProVerif verification summary for of the proposed protocol: Q1–Q9 from the base model (edhoc_m4_hybrid_base.pv) and Q10–Q11 from the key-rotation model (edhoc_m4_hybrid_pcs.pv).

Table 4 compares all seven variants. The classical methods T0–T3 and EAP-TLS Classic reach R1–R5, R7, and R9 only against a classical adversary and fail R6 and R8: in their quantum-setting models, every authentication and secrecy query turns false, whereas all queries of the two hybrid variants keep their classical verdicts. EAP-TLS Hybrid PQC, which uses the same X25519MLKEM768 group and ML-DSA-44 certificates, retains its guarantees against a quantum adversary, consistent with the composition argument for hybrid combiners [20,21] and the EUF-CMA security of ML-DSA-44 [58], and with published TLS 1.3 analyses [60]. Like every other variant, however, it fails post-compromise security, because TLS 1.3 defines no static-key rotation. The T0–T3 and EAP-TLS models follow their specifications exactly, so their ○ on R6 reflects the absence of a rotation step in the standards rather than an inherent impossibility. The proposed protocol is the only variant that is fully secure on eight of the nine requirementsall nine requirements, with forward secrecy and identity concealment conditional on key erasure.

Table 4. ProVerif verification results across EDHOC authentication for all seven methodsand the two EAP-TLS baselines.

Security Requirement	EAP-EDHOC Method 0 [10,12]	EAP-EDHOC Method 1 [10,12]	EAP-EDHOC Method 2 [10,12]	EAP-EDHOC Method 3 [10,12]	EAP-EDHOC Hybrid (Sec. 4)	EAP-TLS Classic [8]	EAP-TLS Hybrid PQC [8,43]
R1 Mutual authentication	◐	◐	◐	◐	●	◐	●
R2 Identity concealment	◐	◐	◐	◐	●†	◐	●
R3 Confidentiality	◐	◐	◐	◐	●	◐	●
R4 Secure key exchange	◐	◐	◐	◐	●	◐	●
R5 Forward secrecy	◐	◐	◐	◐	●	◐	●
R6 Post-compromise security	○	○	○	○	●	○	○
R7 AEAD protection	◐	◐	◐	◐	●	◐	●
R8 Quantum resilience	○	○	○	○	●	○	●
R9 KCI resistance	◐	◐	◐	◐	●	◐	●

○ not secure; ◐ partial (secure only against a classical adversary); ● fully secure (classical *and* quantum adversary). Every entry is a ProVerif result, except the ○ marks on R6 for the six baselines, whose specifications define no static-key rotation. †ID_CRED_I is sent in `Message1` under a key derived from the DN-AAA's current one-time key. It stays secret after that key is erased, but is exposed if the current sk_B is compromised before the run. The outer EAP identity that selects the DN-AAA record is not covered.

6. Performance Evaluation

6.1. Testbed and Cryptographic Cost

Experiments ran on a Windows 11 Pro host (Intel Core i9-14900K, 64 GB DDR5) in a VirtualBox VM with Ubuntu 24.04 LTS (16 GB RAM, 12 vCPUs). The testbed uses Open5GS [31], UERANSIM [32], and FreeRADIUS [33], all co-located in the VM and communicating over loopback, so the measured network overhead reflects protocol-stack, scheduling, and RADIUS processing rather than RAN/WAN propagation and should be read as a lower bound. We modified Open5GS 2.7.2, UERANSIM 3.2.7, and FreeRADIUS 3.2.6. The EDHOC methods (0–4) use libsodium [62] for X25519, Ed25519, SHA-256, and HKDF, mbedTLS [63] for AES-128-GCM (both from the Ubuntu 24.04 LTS repositories), and the PQClean [64] clean reference implementations of ML-KEM-768 and ML-DSA-44. The EAP-TLS baselines use OpenSSL: EAP-TLS Classic runs on the system OpenSSL 3.0.13, while EAP-TLS Hybrid PQC runs on a separately built OpenSSL 3.2.6 with liboqs 0.16 [65] and oqs-provider 0.12 [66], because ML-DSA-44 certificates in TLS 1.3 require OpenSSL 3.2 or later; each Hybrid PQC handshake is checked at run time to have negotiated the X25519MLKEM768 group and an ML-DSA-44 certificate. Both baselines follow EAP-TLS 1.3 [8]. The OpenSSL builds implement TLS 1.3 as specified in RFC 8446, which RFC 9846 [9] has since obsoleted with a backward-compatible revision that leaves the handshake messages and their signatures unchanged; RFC 9846 also forbids reusing a key share across connections, so every handshake generates fresh shares, as counted in Table 6. X25519MLKEM768 is the hybrid group of RFC 10024 [43], and the ML-DSA-44 certificates follow RFC 9881 [67] and the ML-DSA profile for TLS 1.3 [44]. In both baselines each party signs one `CertificateVerify` and verifies the peer's `CertificateVerify` and the CA signature on the peer's certificate. The source code, configuration, and benchmark scripts are publicly available [34]. ~~EAP-EDHOC runs over real~~ Figure 6 shows the components we modified or added. On the full 5G path, the UE (UERANSIM `nr-ue`) runs the EDHOC initiator inside NAS PDU Session Authentication messages, the AMF forwards these as DL NAS Transport, and the SMF acts as the EAP authenticator within PDU session establishment: it hands each EAP payload, without blocking its event loop, to a RADIUS client co-located with the UPF over persistent sockets, which speaks EAP-over-RADIUS (RFC 2865/3579) [23,24] to a FreeRADIUS `rlm_eap_edhoc` responder ~~along the path UE → AMF → SMF → UPF → FreeRADIUS. All~~ acting as the DN-AAA. The FreeRADIUS module implements all five EDHOC methods~~are implemented~~: methods 0–3 use three messages and method 4 (the proposal) four messages with EAP fragmentation. The UE, the responder, and the benchmark client link the same EDHOC library. We measure on two paths. The method comparison uses a RADIUS benchmark client that acts as both the EAP peer and the RADIUS client against the same FreeRADIUS server, so all seven methods, including the two EAP-TLS ~~Classic and EAP-TLS Hybrid PQC are benchmarked end to end on the same testbed~~baselines for which UERANSIM has no peer, are measured in the same way; this path covers the RADIUS and DN-AAA segment and leaves out the 5G NAS transport. The proposal is also run through the full path UE → gNB → AMF → SMF → UPF → FreeRADIUS (Section 6.3). On this path we enabled `SCTP_NODELAY` on the NGAP socket of UERANSIM, as Open5GS already does on its side; without it, Nagle's algorithm held the gNB's uplink NGAP messages until the AMF's delayed acknowledgement and added about 160 ms to every PDU session establishment after the first.

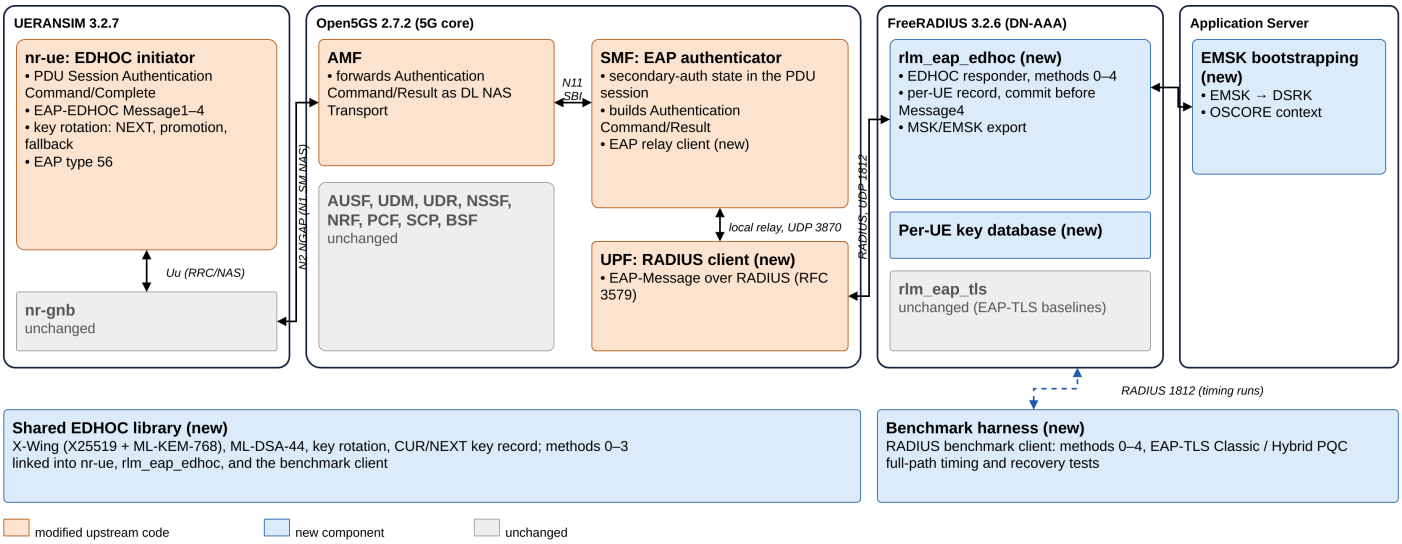


Figure 6. Testbed components: modified, new, and unchanged.

During the preparation of this manuscript, the authors used Claude (Anthropic; model claude-opus-5-5) to convert the manuscript to the journal template and to condense and restructure the text; the tool was not used to generate data, results, or analyses. The authors reviewed and edited the output and take full responsibility for the content of this publication.

Table 5 lists primitive costs over 3000 iterations. The most expensive operation, ML-DSA-44 signing, averages ~~0.238~~0.223 ms, and all ML-KEM-768 operations stay below 0.05 ms. Table 6 counts the operations that each party performs in each of the seven methods, and Table 7 groups the resulting compute time by function. We count every key generation inside the handshake for all methods, including the next one-time key pair of the proposal. A party could generate that key pair in advance, just as a TLS client could generate its key share, but common stacks, including the OpenSSL stack we measure, do neither, so we compare both protocols as they run. Per handshake, the ~~proposed method costs 1.259~~proposal costs 0.985 ms, split ~~almost~~ exactly evenly between the initiator ~~(0.651 ms) and the responder (0.608 ms), a direct consequence of the role symmetry of Section 4.5. The classical methods 0–3 cost 0.148–0.180 ms, and~~ and the responder (0.492 ms each), because the two parties run mirror-image operation sets: one X-Wing encapsulation, one decapsulation, one key-pair generation, one signature, and one verification each. Measured in process over 5 × 2,000 handshakes, which adds the key schedule, AEAD, and serialisation, the two parties together spend 1.131 ms; following the X-Wing specification exactly, with a decapsulation that takes pk_X from the stored public key, saved 0.045 ms against our earlier implementation. The EAP-TLS ~~Classic and Hybrid PQC 0.180 ms and 0.820 ms~~figures include, besides the `CertificateVerify` signature, the verification of the CA signature on the peer's end-entity certificate that each side performs (the chain has no intermediate CA; the CA signs with Ed25519 and ML-DSA-44, respectively), whereas EAP-EDHOC uses provisioned verification keys and verifies no certificate. PQC thus multiplies computation by about ~~7~~4.9–~~8~~×5.4× for EAP-EDHOC and 3.7× for EAP-TLS, yet the absolute cost stays ~~under 1.5~~below 1 ms. Figures 7 and 8 break down computation and communication for all seven methods. In the right panel of Figure 7, "Initiator" is the UE's local processing (for the proposal, its key commit after EAP-Success lies outside this time) and "Responder" is the time the UE waits for replies, including the RADIUS round-trips, the DN-AAA's processing, and its key storage.

Table 5. Mean latency of cryptographic primitives (3000 iterations).

Operation	Mean (ms)	StdDev (ms)	95% CI (±ms)
Keygen_X25519	~~0.024004~~ 0.025535	~~0.028271~~ 0.029916	~~0.001012~~ 0.001071
ScalarMult_X25519	~~0.024889~~ 0.025164	~~0.016870~~ 0.019026	~~0.000604~~ 0.000681
Keygen_Ed25519	~~0.010882~~ 0.010730	~~0.012333~~ 0.017312	~~0.000441~~ 0.000619
Signature_Ed25519	~~0.011226~~ 0.011070	~~0.016348~~ 0.010091	~~0.000585~~ 0.000361
Verify_Ed25519	~~0.029693~~ 0.029439	~~0.021514~~ 0.020345	~~0.000770~~ 0.000728
SHA256	~~0.000263~~ 0.000300	~~0.000003~~ 0.001233	~~0.000000~~ 0.000044
HKDF_SHA256	~~0.001448~~ 0.001456	~~0.005969~~ 0.005022	~~0.000214~~ 0.000180
AES128GCM	~~0.000255~~ 0.000226	~~0.001266~~ 0.000021	~~0.000045~~ 0.000001
Keygen_MLKEM768	~~0.034582~~ 0.032440	~~0.019357~~ 0.024176	~~0.000693~~ 0.000865
Encaps_MLKEM768	~~0.039528~~ 0.033775	~~0.047955~~ 0.019317	~~0.001716~~ 0.000691
Decaps_MLKEM768	~~0.047708~~ 0.041870	~~0.034353~~ 0.021982	~~0.001229~~ 0.000787
Keygen_MLDSA44	~~0.057264~~ 0.056854	~~0.039540~~ 0.041449	~~0.001415~~ 0.001483
Signature_MLDSA44	~~0.237660~~ 0.223241	~~0.203541~~ 0.212971	~~0.007284~~ 0.007621
Verify_MLDSA44	~~0.062323~~ 0.059596	~~0.041196~~ 0.028310	~~0.001474~~ 0.001013

Table 6. Cryptographic operations (Initiator/Responder) and compute time per handshake.

	X25519		ML-KEM-768						Compute (ms)		
Method	**KeyGen**	**DH**	**KeyGen**	**Encaps**	**Decaps**	**Sign**	**Verify**	**KeyGen**[b]	**I**	**R**	**Total**
Classical level (X25519, Ed25519)											
EAP-EDHOC Method 0	1/1	1/1	–	–	–	1/1	1/1	2	0.091	0.091	0.182
EAP-EDHOC Method 1	1/1	2/2	–	–	–	1/0	0/1	2	0.087	0.105	0.192
EAP-EDHOC Method 2	1/1	2/2	–	–	–	0/1	1/0	2	0.105	0.087	0.192
EAP-EDHOC Method 3	1/1	3/3	–	–	–	0/0	0/0	2	0.101	0.101	0.202
EAP-TLS Classic	1/1	1/1	–	–	–	1/1	2/2	2	0.121	0.121	0.241
Hybrid PQC level (X25519 + ML-KEM-768, ML-DSA-44)											
Proposal	2/2[a]	2/2	1/1[a]	1/1	1/1	1/1	1/1	6	0.492	0.492	0.985
EAP-TLS Hybrid PQC	1/1	1/1	1/0	0/1	1/0	1/1	2/2	3	0.467	0.427	0.894

Signatures use Ed25519 (classical) and ML-DSA-44 (hybrid PQC); for EAP-TLS each verification count is `CertificateVerify` + certificate. Compute times follow from Table 5. [a] Per party, one X25519 key pair is the ephemeral share inside X-Wing encapsulation; the other X25519 key pair and the ML-KEM-768 key pair form the next one-time X-Wing key pair (key rotation). In Methods 1–3 a static-DH authentication costs one DH on *each* side. [b] Key generations per handshake, both parties (X25519 and ML-KEM-768).

Table 7. Compute per handshake by function (ms).

Method	**Key Exchange**	**Authentication**	**Certificate Check**	**Key Rotation**	**Total**
Classical level					
EAP-EDHOC Method 0	0.101	0.081	–	–	0.182
EAP-EDHOC Methods 1, 2	0.101	0.091	–	–	0.192
EAP-EDHOC Method 3	0.101	0.101	–	–	0.202
EAP-TLS Classic	0.101	0.081	0.059	–	0.241
Hybrid PQC level					
Proposal	0.303	0.566	–	0.116	0.985
EAP-TLS Hybrid PQC	0.209	0.566	0.119	–	0.894

Both parties, from Table 6. Key exchange: ephemeral X25519 shares, ephemeral DH, and KEM operations. Authentication: transcript signatures and their verification, or static DH for the MAC-based methods. Certificate check: CA signature on the peer's certificate. Key rotation: next one-time X-Wing key pair.

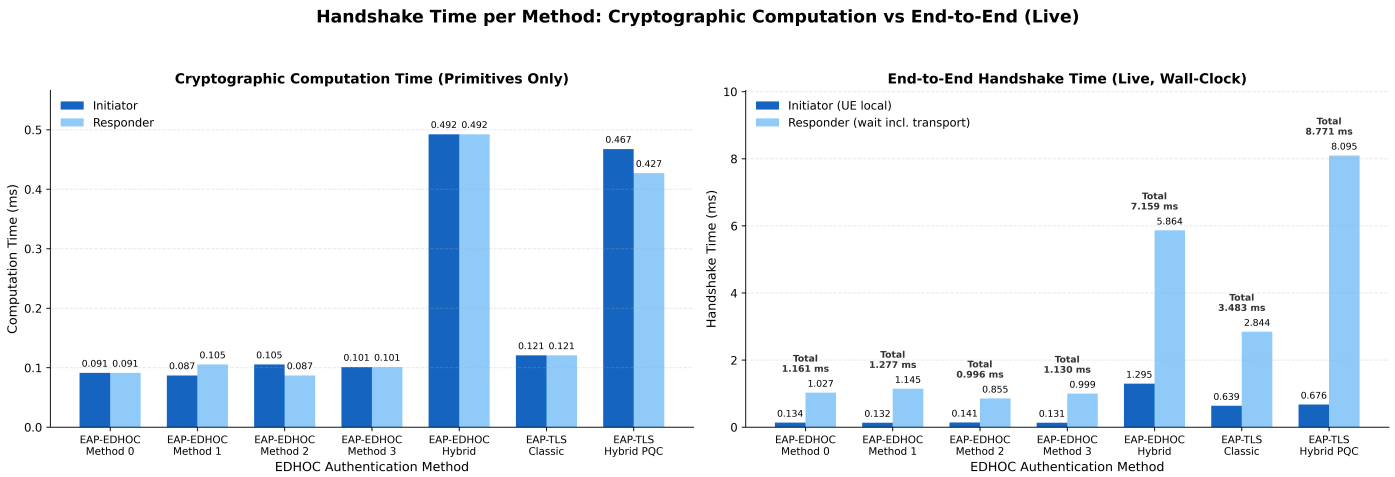


Figure 7. ~~Per-method handshake computation~~ Computation cost (**left**) and end-to-end handshake time (**right**).

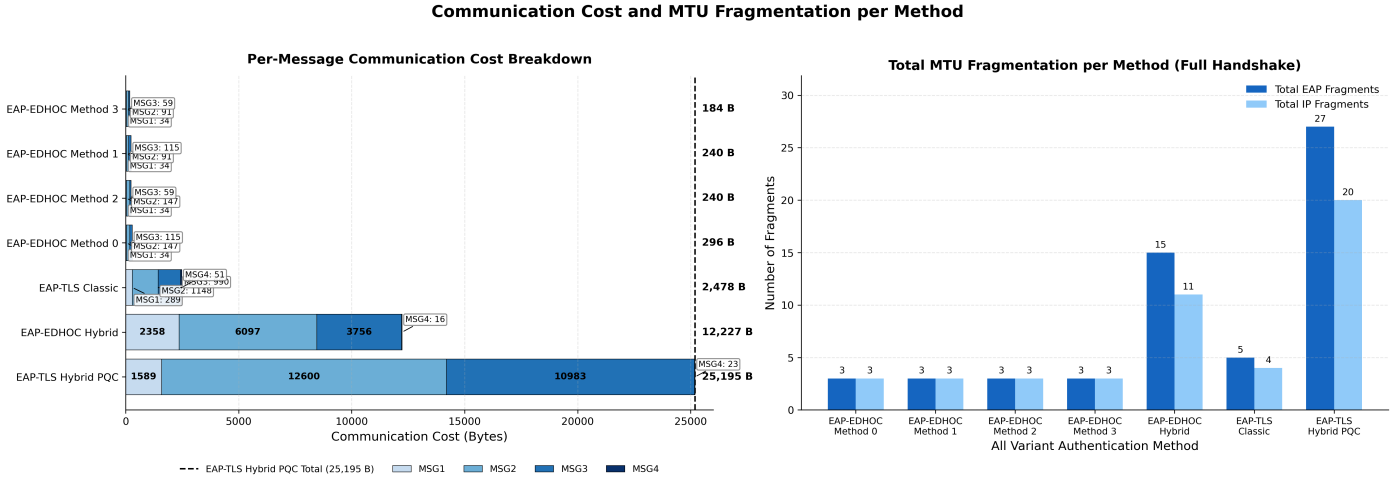


Figure 8. ~~Per-method communication (message~~ Message size ~~) breakdown~~ and MTU fragmentation.

6.2. End-to-End Latency, Message Size, and Framing

Over the live EAP-over-RADIUS path (~~100~~ 200 iterations per method in two runs, no loss), the proposed method completes in ~~7.492~~7.159 ms on average (median ~~5.410 ms, p95 14.226~~6.582 ms)~~with a 100% success rate, against 0.821~~, against 0.996–~~0.992~~1.277 ms for methods 0–3 ~~, 2.835~~(medians 0.954–1.017 ms; a pooled run of 10,000 handshakes per method in two opposite orders on the same idle machine gives medians of 1.021–1.039 ms, which we consider the more reliable estimate), 3.483 ms for EAP-TLS Classic ~~, and 9.241~~(median 3.101 ms), and 8.771 ms for EAP-TLS Hybrid PQC ~~. The gap is driven by fragmentation, not computation~~(median 8.620 ms). Each run of a method was preceded by 20 discarded warm-up handshakes. Absolute times vary between sessions of our VM (EAP-TLS Hybrid PQC between 8.8 and 11.8 ms across our measurement sessions), so we compare methods only within one session. Two costs separate the proposal from methods 0–3, and neither is computation. The first is fragmentation: the proposal needs ~~17~~ 15 RADIUS round-trips versus 3 for methods 0–3, and EAP-TLS Classic and Hybrid PQC need 6 and 28. The second is durable key rotation. Two `fdatasync` calls lie on the critical path, the UE's write of NEXT before `Message3` and the DN-AAA's commit before `Message4` (Section 4.4); the two writes that zero the old secret keys follow `Message4` and lie outside the measured time (the UE's promotion takes 0.763 ms on average). With `strace -w`, one `fdatasync` takes 0.74–0.80 ms of wall-clock time on the virtual disk of

our VM, including tracing overhead. To isolate the storage cost, we ran the proposal in 6 alternating rounds of 100 handshakes with the key directories on the VM disk and on a RAM-backed `tmpfs`, where a flush is almost free: the medians are 6.688 and 5.527 ms, so durable storage costs 1.16 ms (bootstrap 95% interval [1.06, 1.29] ms). A separate four-way ablation that moves only one party's keys to `tmpfs` attributes 0.58 ms ([0.47, 0.68] ms) to the UE's write-ahead and 0.51 ms ([0.39, 0.60] ms) to the DN-AAA's commit. Our first implementation stored each key in its own file (write, `fsync`, atomic rename) and erased old keys with separate flushes, four `fsync` and two `fdatasync` calls per handshake, which added 7.9 ms (13.223 ms against 5.314 ms on `tmpfs`). Storage with a power-loss-protected write cache completes a flush much faster than our virtual disk, so this share is specific to the testbed. Generating the next key pair in advance would be an optional deployment optimisation: it lowers the in-process computation from 1.131 to 1.020 ms, and in an earlier session 8 alternating rounds of 100 handshakes showed that it lowered the live median by about 0.12 ms (6.139 vs. 6.016 ms; bootstrap 95% interval $[-0.03, 0.22]$ ms), the same order as the run-to-run variation of our testbed, so all reported results generate it inside the handshake.

The message sizes explain this. Classical messages fit in one EAP fragment (34–147 bytes). In the proposed method, each X-Wing element adds 1.1–1.2 KB and each ML-DSA-44 signature about 2.4 KB~~, so~~ . `Message1` (~~≈ 2.35 KB~~2,358 B: ct_B and the encrypted `Plaintext1` with Npk_A), `Message2` (~~≈ 6.0 KB) ,~~ 6,097 B) and `Message3` (~~≈ 5.0 KB~~3,756 B) exceed the 1500-byte MTU and are split into 3, 7, and 4 EAP fragments (RFC 3579, 253-byte attributes) [24] and ~~IP fragments, while~~ into IP fragments; `Message4` (~~≈ 1.2 KB~~16 B, essentially an AEAD tag) fits in one. ~~Although individual messages differ widely in size, the~~ The handshake totals 12,227 B in 15 EAP fragments. The load per direction is ~~nearly~~ balanced: the UE sends ~~about 7.35 KB~~ 6,114 B (`Message1+Message3`) and the DN-AAA ~~about 7.2 KB~~ 6,113 B (`Message2+Message4`). Announcing the next keys in `Message1` and `Message2` rather than in `Message3` and `Message4` leaves the total unchanged but costs one more fragment, because `Message1` no longer fits in one MTU. EAP-TLS Classic totals 2,478 B in 5 EAP fragments and EAP-TLS Hybrid PQC 25,195 B in 27, the most of any method. Cross-implementation checks against pyca/cryptography over OpenSSL passed all 11 tests, with sizes matching the standards (ML-KEM-768 public key 1184 B, ciphertext 1088 B; ML-DSA-44 key 1312 B, signature 2420 B).

Because all components share one VM, the round-trip time (RTT) between the SMF and the DN-AAA is close to zero, which understates the cost of fragmentation. Table 8 projects the measured loss-free means to a non-zero RTT with $T(\mathrm{RTT}) \approx T_{\mathrm{loopback}} + n_{\mathrm{RT}} \cdot \mathrm{RTT}$, where n_{RT} is the measured number of RADIUS round-trips. Under this model the proposal, which already completes in less time on loopback and needs about half the round-trips, stays below EAP-TLS Hybrid PQC at every RTT, and the ratio grows from 1.2 at 0 ms to 1.8 at 10 ms (157 vs. 289 ms); queuing, congestion, and per-packet processing on a real path can shift these values.

Table 8. Projected handshake time versus SMF–DN-AAA RTT (ms).

Method	n_{RT}	0 ms	1 ms	5 ms	10 ms	20 ms
EAP-EDHOC Methods 0–3 (median)	3	1.0	4.0	16.0	31.0	61.0
EAP-TLS Classic	6	3.5	9.5	33.5	63.5	123.5
Proposal (durable key storage)	15	7.2	22.2	82.2	157.2	307.2
EAP-TLS Hybrid PQC	28	8.8	36.8	148.8	288.8	568.8

$T_{\mathrm{loopback}} + n_{\mathrm{RT}} \cdot \mathrm{RTT}$, from the measured loss-free means and round-trip counts; for Methods 0–3, T_{loopback} is the median of the pooled 10,000-handshake runs.

The two panels of Figure 7 rank the methods differently~~. In computation, the proposal is the most expensive (1.259 ms~~ , so we compare each EAP-EDHOC variant with the EAP-TLS baseline at the same security level (Tables 6 and 7). *Classical pair* (Methods 0–3 vs. ~~0.820 ms for~~ EAP-TLS ~~Hybrid PQC)because its four messages run more X25519~~ Classic, both Ed25519 and ~~ML-KEM-768 operations; both perform the same signatures. End to end, the order reverses (9.241~~X25519): both run the same ephemeral X25519 exchange (0.101 ms) and spend a similar amount on authentication (0.081–0.101 ms), while each EAP-TLS party also verifies one certificate signature (0.059 ms in total; we used Ed25519 certificates instead of RSA-2048 so that both sides of the pair use the same primitives). EAP-EDHOC therefore computes slightly less (0.182–0.202 ms vs. 0.241 ms), sends 8–13× fewer bytes (184–296 B vs. 2,478 B), needs half the round-trips (3 vs. 6), and completes 3.0–3.3× sooner in the median on our testbed (0.954–1.017 ms vs. ~~7.492~~3.101 ms)~~because~~ . *Hybrid PQC pair* (the proposal vs. EAP-TLS Hybrid PQC, both X25519+ML-KEM-768 and ML-DSA-44): here the proposal computes more (0.985 vs. 0.894 ms). Both spend the same 0.566 ms on two ML-DSA-44 signatures and two verifications of the transcript. The proposal spends 0.094 ms more on key exchange, because it runs one X-Wing encapsulation per direction instead of the single hybrid exchange of TLS, and 0.116 ms on the next key pairs of both parties, which is the price of forward secrecy and post-compromise security without an ephemeral key pair; EAP-TLS in turn spends 0.119 ms on certificate checks. In transport the order reverses: EAP-TLS carries ML-DSA-44 inside an X.509 chain in a TLS record ~~,~~whereas EAP-EDHOC carries a compact detached CBOR/COSE value~~: 25,195~~, so the proposal sends half the bytes and needs about half the round-trips (12,227 B and ~~28 round-trips against 14,561~~15 vs. 25,195 B and ~~17. The same crossover appears classically: EAP-TLS Classic and Method 0 have identical compute cost (0.180 ms , both Ed25519 and one X25519 exchange; we used an Ed25519 certificate instead of RSA-2048 to isolate this effect), yet EAP-TLS Classic is 2.9× slower end to end (2.835~~ 28). On our loopback testbed it completes in less time (7.159 vs. ~~0.992 ms) due to TLS~~8.771 ms mean, 6.582 vs. 8.620 ms median, a factor of 1.2–1.3), although it stores its rotated keys durably in every run and EAP-TLS stores none. Subtracting computation and storage, each RADIUS round-trip costs about 0.3 ms there for both protocols, so the 13 round-trips the proposal saves outweigh its 0.09 ms of extra computation and its two `fdatasync` calls (1.16 ms on our VM), by a margin that grows with the RTT (Table 8). The `tmpfs` figure is only a reference point, since `tmpfs` is not crash-safe and the protocol's forward secrecy relies on durable rotation. Across both pairs the compute difference stays below 0.1 ms, while EDHOC's compact encoding keeps its transport cost below that of EAP-TLS; the comparison favours EAP-EDHOC where round-trips are costly, not because its cryptography is cheaper.

6.3. *Full 5G Core Path*

To confirm that the proposal runs over a complete 5G core, we ran it through UERANSIM and Open5GS with microsecond timestamps (`CLOCK_MONOTONIC`) in the UE, the SMF, the UPF relay, and the DN-AAA. The SMF runs secondary authentication as step 6 of PDU session establishment in TS 23.502 [68]: after the SM context is created and the subscription data are retrieved, and before policy association, UPF selection, and N4 session establishment. It sends an EAP-Request/~~X.509 framing and 6~~ Identity in a PDU Session Authentication Command; the UE answers with a DN-specific identity rather than its SUPI, and the DN-AAA selects the UE's key record by this identity. On success the EAP-Success travels in the PDU Session Establishment Accept; on failure the SMF sends a PDU Session Establishment Reject with cause #29 carrying the EAP-Failure [69] . A packet capture of one session confirms this order. Three series of 100 PDU sessions

with secondary authentication alternated with three series without it; each series restarted the stack and discarded five warm-up sessions. All 300 sessions were accepted with EAP-Success, both parties derived the same MSK in every session (compared through a truncated hash), and both rotated their keys every time. Message sizes equal those of the benchmark client, with 15 NAS and 15 RADIUS round-trips. Table 9 splits the time into the RADIUS segment, the relay, and the 5G core transport through the UE, gNB, AMF, and SMF.

Table 9. Proposal over the full 5G core path (300 sessions, ms).

Segment	Median	95% CI	Mean	p95
Added to PDU session establishment[a]	93.4	[90.4, 95.6]	117.8	–
Secondary authentication at the UE[b]	90.0	[88.0, 92.5]	115.0	183.9
5G core transport (UE, gNB, AMF, SMF)	82.7	[81.9, 85.2]	106.9	167.0
SMF ↔ UPF relay hop	3.6	[3.5, 3.7]	4.6	8.5
RADIUS processing in the UPF relay	1.6	[1.5, 1.7]	2.3	5.7
DN-AAA (FreeRADIUS, from the UPF relay)	6.5	[6.3, 6.8]	8.0	14.4
UE write of NEXT (before `Message3`)	1.37	[1.07, 2.05]	1.91	3.65
DN-AAA commit (before `Message4`)	0.71	[0.69, 0.72]	0.93	1.79
UE promotion (after `Message4`)	0.63	[0.61, 0.65]	0.83	1.93

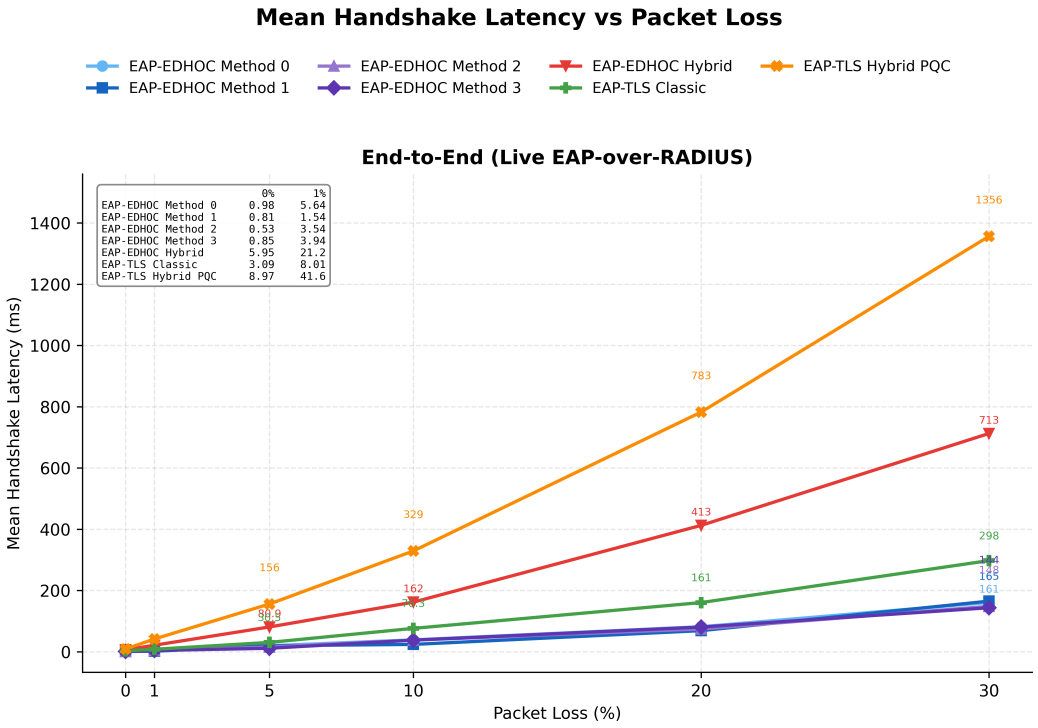
[a] PDU Session Establishment Request to Accept, with minus without secondary authentication. [b] First PDU Session Authentication Command to the Accept carrying EAP-Success; the 5G core transport row includes the policy, UPF, and N4 steps after the Access-Accept (about 5 ms), which also occur without secondary authentication. Median CIs by bootstrap (4000 resamples).

Secondary authentication adds 93 ms in median to PDU session establishment, which takes 21 ms without it and 114 ms with it. The 5G core transport dominates, at about 5.5 ms per NAS round-trip in UERANSIM and Open5GS, so on this path every round-trip the proposal saves weighs more than on the RADIUS segment alone. The DN-AAA takes 6.5 ms seen from the UPF relay. The benchmark client waits 5.3 ms for it in the median, both on an otherwise idle VM and with the 5G core running but idle, so the remaining 1.2 ms appear only while the UE and the core are active between round-trips. ~~EDHOC's compact encoding, not its primitives, keeps it below~~ The relay adds 3.6 ms for the SMF–UPF hop and 1.6 ms for building and verifying the RADIUS packets, over 15 round-trips. The first series was slower than the other two (median 98 against 86–89 ms) and the tail is long, so we report medians. Dropping `Message3` or `Message4` once at the UPF relay (10 runs each) reproduced the recovery of Section 4.4 on the full path. The session with the drop timed out and was rejected without an EAP-Failure, so the UE kept its keys. After a lost `Message3`, the next session started on NEXT and was rejected with cause #29 and an EAP-Failure, and the session after it succeeded on the old keys (median 1.42 s from the first recovery request; recovery thus costs one extra PDU session and the 1 s `reject_delay` of FreeRADIUS). After a lost `Message4`, the next session succeeded directly on NEXT (median 125 ms). No run desynchronised, and none required re-provisioning. Because UERANSIM has no EAP-TLS ~~at every security level, and the gap widens with PQC~~peer, the comparison with EAP-TLS stays on the RADIUS path.

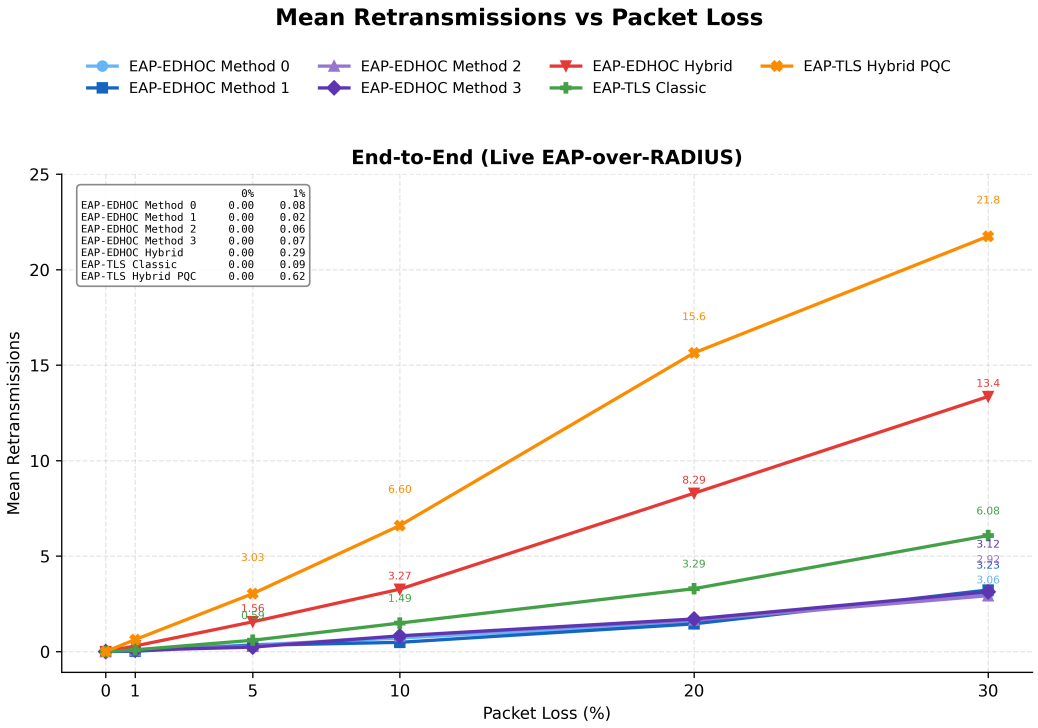
6.4. Behaviour under Packet Loss

Figures 9 and 10 report behaviour under emulated application-level loss with EAP-style retransmission (RTO 40 ms, up to 6 retransmissions per exchange, 120 trials per cell) for all seven methods. Methods 0–3 keep a 100% success rate up to ~~20% loss, while the larger-payload methods degrade earlier: the proposal and~~ 10% loss, reach 99.2–100% at 20% and 96.7–98.3% at 30%. EAP-TLS Classic ~~reach~~ reaches 100% at 20% and 96.7% at 30% loss, and EAP-TLS Hybrid PQC 99.2% and 79.2%. The proposal reaches 98.3% at 20% loss (mean 412.6 ms, 8.3 retransmissions) and 93.3% at 30% (mean 712.6 ms, 13.4

retransmissions averaged over all trials), between the two EAP-TLS baselines at 30%, in line with its 15 EAP fragments against their 5 and 27. With 120 trials per cell the success rates carry Wilson 95% intervals of about ±6 points (e.g., [87.4, 96.6]% for the proposal and [71.1, 85.5]% for EAP-TLS Hybrid PQC ~~96.7% at~~ at 30% loss); at 30% the difference between the two success rates is significant (Fisher's exact test, $p = 0.002$), while up to 20% loss the two rates do not differ ($p = 1.0$). We read them together with a simple model: if one round-trip succeeds with probability $p = (1 - L)^2$ and an exchange fails after 7 attempts, a handshake with n_{RT} round-trips succeeds with probability $(1 - (1 - p)^7)^{n_{RT}}$. At 30% loss ~~, the classical methods remain at 95.8~~this predicts 87.4% for the proposal and 77.7% for EAP-TLS Hybrid PQC, against the measured 93.3% and 79.2%, and about 15.6 and 29.1 retransmissions per successful handshake (measured: 14.3 and 27.5 among successful handshakes) at 40 ms each (≈ 624 and 1,166 ms against the measured 713 and 1,357 ms). The loss behaviour is therefore governed by the number of round-trips; latency differs at every loss level, success rate only at the highest. Within each security level the EAP-EDHOC variant, which needs fewer round-trips, degrades less in our emulation: at 30% loss, Methods 0–3 average 144 ~~–97.5%,~~ 165 ms against 298 ms for EAP-TLS Classic~~at 95.8%, the proposal drops to 87.5%, and~~ , and the proposal 713 ms against 1,357 ms for EAP-TLS Hybrid PQC~~to 73.3%~~. The proposal has the lower mean latency of the hybrid pair at every loss level we tested, and the absolute gap widens with loss (21.2 vs. 41.6 ms at 1%, 80.9 vs. 155.9 ms at 5%). We also exercised the recovery of Section 4.4 on the live stack. With `Message3` dropped deterministically (20 runs), the next run started from NEXT, was rejected, and completed after one fallback to the old keys; with `Message4` dropped (20 runs), the next run completed directly with NEXT. No run desynchronised, and none required re-provisioning. The fallback run took 1,011 ms because FreeRADIUS delays every reject by 1 s by default (`reject_delay`); with this delay disabled it took 9.7 ms, and a run after a lost `Message4` took 8.2 ms. In a further 150 handshakes at 20% loss, 149 succeeded, and all 30 subsequent loss-free handshakes succeeded without re-provisioning. Each extra fragment is an independent chance of loss, so latency and retransmissions grow with fragment count~~: the proposal reaches 409.3 ms and 9.3 retransmissions at 20% loss and 724.6 ms and 14.7 at 30%, while EAP-TLS Hybrid PQC reaches 1201.8 ms and 20.4 at 30%~~. Fragmentation, not computation, is the main scalability concern.

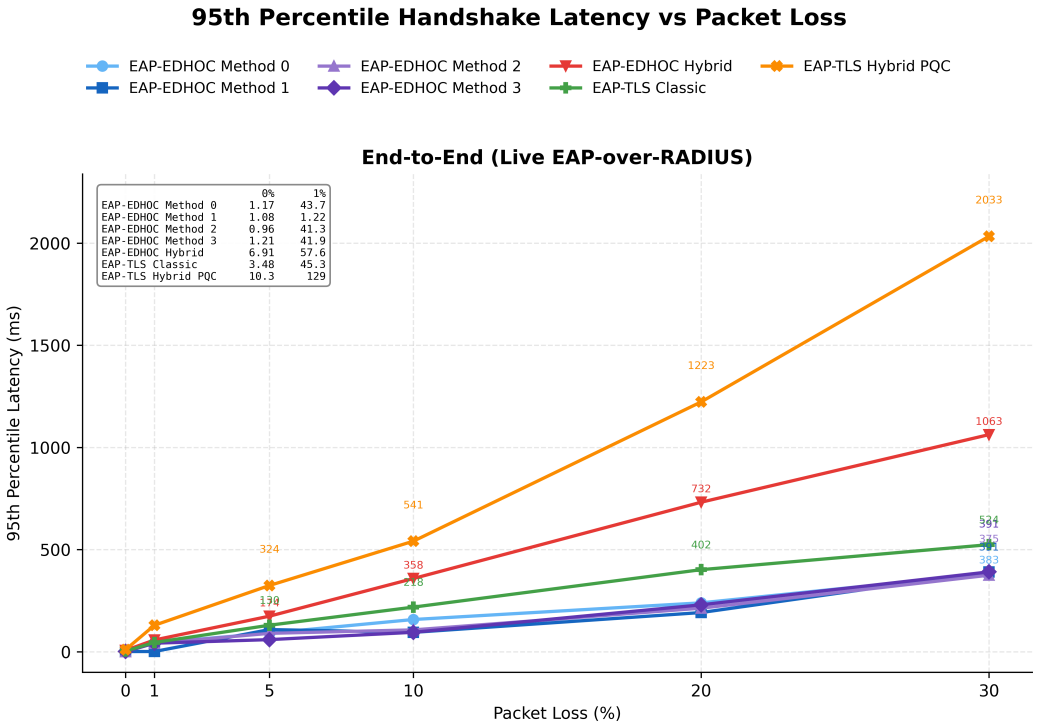


(**a**) Mean latency.

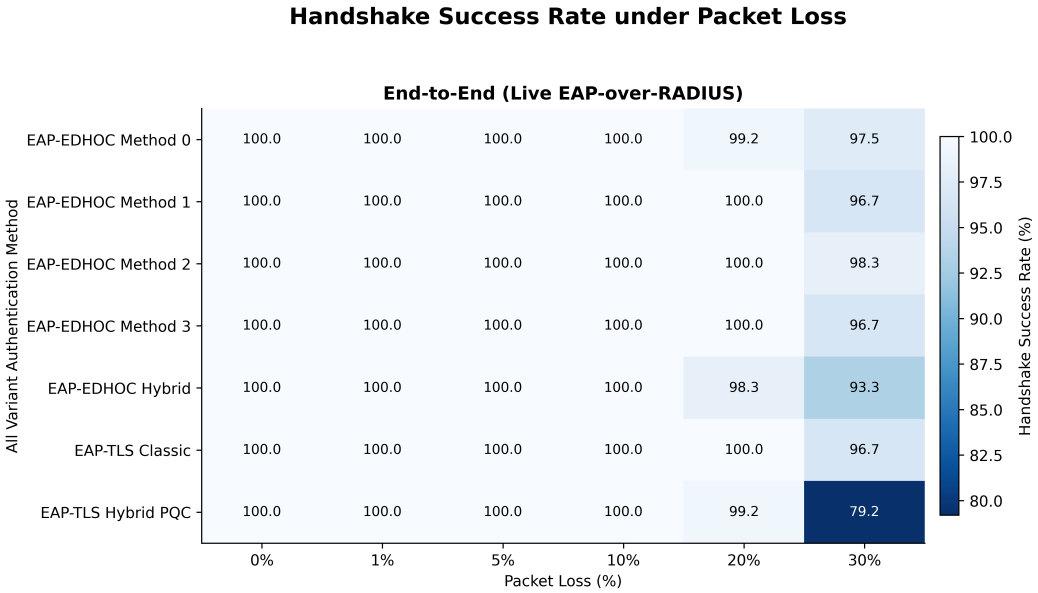


(**b**) Mean retransmissions.

Figure 9. Mean behaviour under packet loss: (**a**) ~~end-to-end~~ latency; (**b**) retransmissions.



(**a**) 95th-percentile latency.



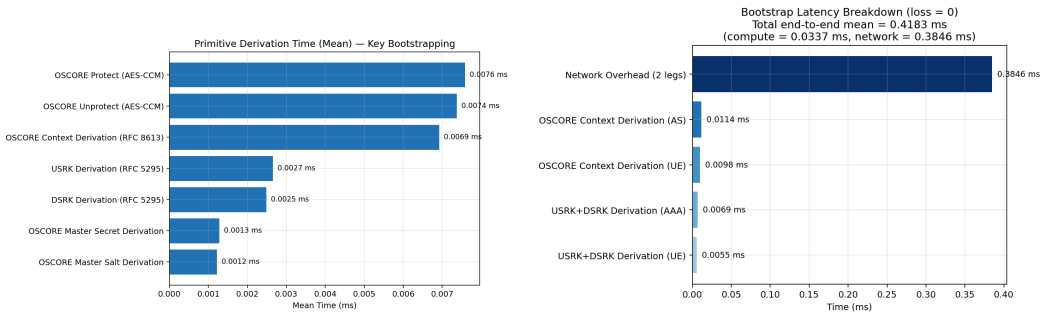
(**b**) Success rate.

Figure 10. ~~Tail latency and reliability under packet~~ Packet loss: (**a**) 95th-percentile latency; (**b**) ~~success-rate heatmap~~success rate.

6.5. EMSK Key-Bootstrapping Cost

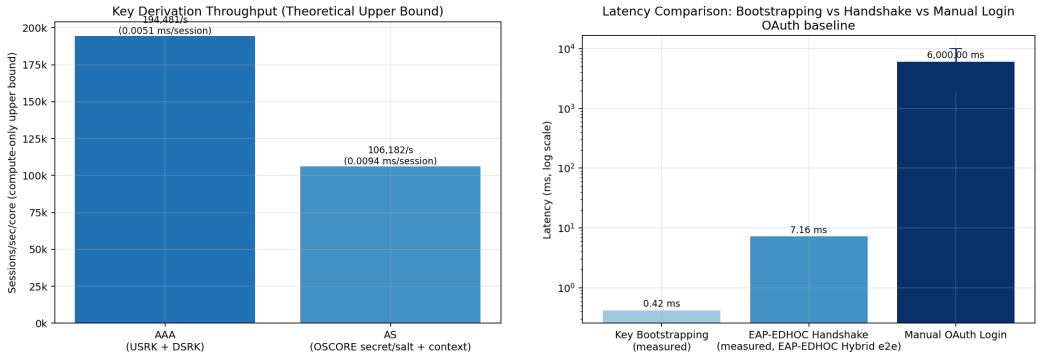
Every primitive in the bootstrapping chain is inexpensive (Figure 11a), ranging from 0.0012 ms (OSCORE Master ~~Secret/~~Salt derivation) to ~~0.0079~~0.0076 ms (OSCORE Protect, AES-CCM). Of the loss-free end-to-end bootstrap latency of ~~0.4142~~0.4183 ms (Figure 11b), only ~~0.0277~~0.0337 ms is spent on the four derivation steps (USRK+DSRK on the UE and AAA sides, OSCORE context on the UE and AS sides); the remaining ~~0.3865~~0.3846 ms is network overhead on the two new legs, AAA→AS (DSRK transport) and UE→AS (first OSCORE contact). The single-core throughput bound (Figure 12a) is

~~203,455~~ 194,481 sessions/s (~~4.915~~5.142 µs/session) on the AAA side and ~~113,289~~ 106,182 sessions/s (~~8.827~~9.418 µs/session) on the AS side, so bootstrapping does not shift the bottleneck away from the AAA server. At ~~0.41 ms~~ 0.42 ms (5.8% of the handshake), bootstrapping is ~~about 18×~~ far cheaper than the ~~7.49 mshandshake~~ handshake (7.159 ms) and far below the 2–10 s typically reported for a manual OAuth 2.0 authorization-code login, at least ~~4,800×~~ 4,700× faster than the 2 s lower bound (Figure 12b).



(**a**) (**b**)

Figure 11. EMSK bootstrapping cost: (**a**) ~~mean latency of each primitive~~primitives; (**b**) ~~compute and network breakdown of the~~ end-to-end ~~bootstrap~~breakdown.



(**a**) (**b**)

Figure 12. EMSK bootstrapping~~scalability~~: (**a**) ~~single-core~~ throughput; (**b**) comparison with ~~the~~ handshake and ~~a manual~~ login.

Two implementation notes apply. The label `"dsrk@ietf.org"` is the reserved label of RFC 5295 Section 4, whereas `"oscore-bootstrap@<specorg>"` for the *ORK* step is a proposed private-use label (RFC 5295 Section 8), not an IANA registration. Cryptographic separation holds because every level uses distinct labels on the same Expand/Extract function: compromising *ORK* in one domain does not reveal another domain's *ORK*, and *DSRK* does not reveal *EMSK* (assuming HMAC-SHA-256 is a PRF); *id_context* separates multiple OSCORE sessions to the same AS without a new *ORK*. RFC 5295 defines no *DSRK* transport; RFC 5749 [70] requires integrity, replay protection, and key wrapping, which we meet with mutual TLS 1.3 between the DN-AAA and the AS, as both sit in one trust domain without an AAA proxy. The chain is purely symmetric and independent of the ML-DSA keys, so its security is inherited from the EMSK.

7. Discussion

Symmetric guarantees, asymmetric costs. The results show two distinct symmetry structures. In the *security* dimension the protocol is role-symmetric: both parties are

authenticated by independent signatures, both are KCI-resistant, both rotate ~~their~~ and erase their one-time keys, and ~~the computation is balanced (0.651 vs. 0.608 ms), as is~~ both the computation (0.492 ms per party) and the traffic per direction (~~≈ 7.35~~ 6,114 vs. ~~≈ 7.2 KB)~~ 6,113 B) are balanced. In the *cost* dimension it is strongly asymmetric: computation ~~is 1.259 msof a 7.492~~(0.985 ms) is about 14% of the 7.159 ms handshake, and most of the rest is fragmentation over ~~17~~ 15 RADIUS round-trips; durable storage of the rotated keys costs two `fdatasync` calls on the critical path, about 1.2 ms on the virtual disk of our VM. This decomposition is actionable. Faster PQC primitives would barely change end-to-end latency, whereas the transport cost can be cut today by raising the EAP fragment size or the RADIUS/MTU budget on the SMF–DN-AAA path, without touching the verified cryptography; the storage cost has already been reduced in this way, from 7.9 ms to about 1.2 ms, by keeping each party's keys in a single two-slot file.

The cost of symmetric rotation. Table 6 shows where the extra computation comes from. Per handshake the proposal performs six key generations (four X25519 and two ML-KEM-768) against three for EAP-TLS Hybrid PQC: besides the X25519 share inside its X-Wing encapsulation, each party generates one complete X-Wing key pair for its next session (0.058 ms). EAP-TLS generates one hybrid key share at the client and one X25519 share at the server, and its forward secrecy comes from these ephemeral shares. The proposal moves forward secrecy to the rotated static keys, and because both parties receive an encapsulation, both must rotate; key generation therefore costs twice as much (0.167 vs. 0.084 ms) and accounts for 0.116 ms of the proposal's 0.985 ms. This is the price of role symmetry and of post-compromise security without an ephemeral key pair, and it stays small against the transport cost.

Design choices against the closest prior art. The KEM-based, signature-free method of Fraile et al. [26,27] would save the ML-DSA-44 sign/verify cost (~~≈ 0.238~~≈ 0.223 ms and ~~0.062~~0.060 ms), which is ~~immaterial against a 7.492 ms handshake~~ small against the end-to-end handshake time, while it is precisely the detached signature that gives formally verified KCI resistance independent of the KEM static key (R9). It would also break the explicit-authentication symmetry of Section 4.5, since a KEM authenticates only the decapsulator. Shin et al.'s Hybrid-EDHOC [28] omits Message 4 to save payload~~, yet~~ . In our protocol Message 4 ~~is the smallest message (1,232 B ,~~ carries no key material, since both next public keys travel in `Message1` and `Message2`, but it remains the protected success indication that EAP-EDHOC mandates [12] and the event after which the UE switches to its new keys; at 16 B it costs one EAP fragment ~~;~~ and one round-trip (Figure 8)~~and carries the Responder's half of the symmetric key rotation that yields post-compromise security (R6). Omitting it would barely reduce fragmentation, since `Message2` and `Message3` dominate the 14,561 B, but would lose a property that T0–T3 lack entirely~~.

Mitigating fragmentation. Three mitigations require no protocol change: (i) raising the EAP fragment size and RADIUS attribute budget beyond the RFC 3579 253-byte default where the operator controls both endpoints; (ii) raising the path MTU on the SMF–UPF–DN-AAA segment, which never crosses the public Internet; and (iii) adopting more compact PQC parameter sets or signatures as they mature, which would shrink `Message2` and `Message3`. None of these affect the authentication logic verified in Section 5.

Threats to validity. (1) All components share one VM over loopback, the full-path figures include UERANSIM's simulated radio link and a VM shared by all components, so the network figures are a lower bound; a smaller path MTU or a higher RTT in a distributed deployment ~~would make~~ makes fragmentation more pronounced, which Table 8 quantifies under a linear model that ignores queuing and congestion. (2) Loss is emulated at the application layer rather than with `netem`, so the relative sensitivity of each method generalises better than the absolute milliseconds. (3) ~~100~~ 200 iterations and 120

trials per cell separate methods that differ by an order of magnitude but are modest for tail estimates among methods 0–3, whose differences are tens of microseconds (Table 5). A pooled 10,000-handshake run per method narrows their median confidence intervals to about ±5 µs, but the spread between the fastest and slowest of them is only 18 µs and their ranking does not follow their compute cost, so we do not rank methods 0–3 against each other. (4) The formal analysis is symbolic, and the quantum adversary is captured by a discrete-log oracle rather than a computational model; a computational proof is future work. (5) Trust in the AAA→AS DSRK channel is a co-location/mTLS assumption analogous to ERP [71], outside the ProVerif model. (6) ~~The rotation-bounded exposure of ID_CRED_I (R2) is not formally modelled; an epoch-based model with per-session key state~~ Forward secrecy rests on the rotation and on the erasure of old secret keys; the recovery rules of Section 4.4 preserve it on every path we model and test, but a computational treatment of the full state machine is future work. Post-compromise security heals with the first run after a compromise, as its definition requires: if the current one-time keys of both parties leak before a run, a passive adversary can read that run, and security returns from the next run because the rotated secret keys never leave the devices. (7) Re-authentication and handover, including the fate of DSRK/OSCORE state at the AS, are not evaluated.

8. Conclusions

We proposed and implemented a role-symmetric hybrid post-quantum EAP-EDHOC protocol for 5G secondary authentication. X-Wing (X25519 + ML-KEM-768) provides key agreement through exactly two opposite-direction encapsulations to one-time static keys, detached ML-DSA-44 signatures authenticate each party independently of its KEM ~~static~~ key, and a two-sided ~~key rotation provides~~ rotation with key erasure provides both forward secrecy and post-compromise security without an ephemeral key pair. ProVerif verification across seven methods shows that only the proposal combines quantum resilience (R8), KCI resistance, forward secrecy, and post-compromise security; identity concealment (R2) holds for ID_CRED_I except under compromise of the Responder's current ~~long-term key , an exposure bounded by per-session rotation~~one-time key before the run. Because both next public keys travel in `Message1` and `Message2`, a lost message never forces re-provisioning, and forward secrecy holds on the recovery path. The exported EMSK bootstraps OSCORE keys following RFC 5295 and RFC 8613, so the UE reaches an application server without further login.

On Open5GS, UERANSIM, and FreeRADIUS, the handshake ~~costs under 1.5 ms of balanced computation~~ exchanges 12,227 B, split evenly between the two directions, and completes in ~~7.492~~7.159 ms on average ~~with a 100% success rate without loss~~on our single-VM testbed, against 8.771 ms for EAP-TLS Hybrid PQC, including the durable storage of the rotated keys (two `fdatasync` calls on the critical path). It computes slightly more than EAP-TLS Hybrid PQC (0.985 vs. 0.894 ms), because it encapsulates in both directions and rotates both keys, but needs about half the EAP round-trips (15 vs. 28). Inside PDU session establishment on the full 5G core, secondary authentication adds 93 ms in median, most of it NAS transport rather than DN-AAA processing (6.5 ms). Its cost is asymmetric, concentrated in fragmentation ~~(17 RADIUS round-trips versus 3 classically), which~~ rather than computation; fragmentation dominates under high packet loss. Future work will reduce this overhead through compact PQC parameter sets, EAP fragment-size and MTU tuning, evaluation under mobility and handover with EMSK re-bootstrapping, a computational security proof, and IANA registration of the OSCORE-bootstrap label. Quantum-resistant secondary authentication is thus deployable on existing 5G infrastructure, with security that is symmetric between the parties and a cost that lies in transport rather than cryptography.

Author Contributions: Conceptualization, A.P.S.I., S.S. and I.Y.; methodology, A.P.S.I.; software, A.P.S.I., I.W.A.J.P. and H.K.; formal analysis, A.P.S.I. and Y.K.; investigation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; data curation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; writing original draft preparation, A.P.S.I. and S.S.; writing review and editing, A.P.S.I. and S.S.; supervision, I.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korean government(MSIT) (No. RS-2024-00398312, Development of Quantum Security Based Device Identification and Test Verification Technologies for 5G Non-Public Network)

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code of the modified Open5GS, UERANSIM, and FreeRADIUS components, the EAP-EDHOC implementation, the ProVerif models, the benchmark scripts, and the raw measurement results supporting this study are openly available in the EdhocSecondaryAuth repository at https://github.com/adipanca/EdhocSecondaryAuth [34].

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

3GPP	3rd Generation Partnership Project
5G-AKA	5G Authentication and Key Agreement
AAA	Authentication, Authorization, and Accounting
AEAD	Authenticated Encryption with Associated Data
AMF	Access and Mobility Management Function
AS	Application Server
AUSF	Authentication Server Function
CBOR	Concise Binary Object Representation
COSE	CBOR Object Signing and Encryption
DN	Data Network
DSRK	Domain-Specific Root Key
EAP	Extensible Authentication Protocol
EDHOC	Ephemeral Diffie–Hellman Over COSE
EMSK	Extended Master Session Key
HKDF	HMAC-based Key Derivation Function
KCI	Key-Compromise Impersonation
KEM	Key Encapsulation Mechanism
MEC	Multi-access Edge Computing
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
MSK	Master Session Key
MTU	Maximum Transmission Unit
OSCORE	Object Security for Constrained RESTful Environments
PQC	Post-Quantum Cryptography
RADIUS	Remote Authentication Dial-In User Service
SMF	Session Management Function
UE	User Equipment
UPF	User Plane Function
USRK	Usage-Specific Root Key

References

1. 3GPP TS 33.501. Security architecture and procedures for 5G System. https://www.3gpp.org/DynaReport/33501.htm, 2024. Release 18. Accessed on 5 March 2026.
2. 3GPP TS 23.501. System architecture for the 5G System (5GS). https://www.3gpp.org/DynaReport/23501.htm, 2024. Release 18. Accessed on 5 March 2026.
3. Edris, E.K.K.; Aiash, M.; Loo, J.K.K. Formal Verification and Analysis of Primary Authentication based on 5G-AKA Protocol. In Proceedings of the 3rd International Symposium on 5G Emerging Technologies (5GET), in conjunction with the 4th International Conference on Fog and Mobile Edge Computing (FMEC). IEEE, 2020, pp. 256–261.
4. Borgaonkar, R.; Hirschi, L.; Park, S.; Shaik, A. New Privacy Threat on 3G, 4G, and Upcoming 5G AKA Protocols. *Proceedings on Privacy Enhancing Technologies* **2019**, *2019*, 108–127.
5. IETF RFC 3748. Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3748.html, 2004. Accessed on 5 March 2026.
6. Gong, S.; EL Azzaoui, A.; Cha, J.; Park, J.H. Secure Secondary Authentication Framework for Efficient Mutual Authentication on a 5G Data Network. *Applied Sciences* **2020**, *10*, 727.
7. IETF RFC 5216. The EAP-TLS Authentication Protocol. https://www.rfc-editor.org/rfc/rfc5216.html, 2008. Accessed on 5 March 2026.
8. IETF RFC 9190. EAP-TLS 1.3: Using the Extensible Authentication Protocol with TLS 1.3. https://www.rfc-editor.org/rfc/rfc9190.html, 2022. Accessed on 29 October 2026.
9. IETF RFC 9846. The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc9846.html, 2026. Obsoletes RFC 8446. Accessed on 29 October 2026.
10. IETF RFC 9528. Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9528.html, 2024. Accessed on 1 February 2026.
11. IETF RFC 9529. Traces of Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9529.html, 2024. Accessed on 2 February 2026.
12. IETF Internet-Draft. Extensible Authentication Protocol (EAP) using EDHOC (EAP-EDHOC). https://datatracker.ietf.org/doc/draft-ietf-emu-eap-edhoc/, 2025. Accessed on 5 March 2026.
13. Fedrecheski, G.; Vučinić, M.; Watteyne, T. Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments. In Proceedings of the Wireless Communications and Networking Conference (WCNC) 2024. IEEE, 2024.
14. Pérez, E.L.; Selander, G.; Mattsson, J.P.; Watteyne, T.; Vučinić, M. EDHOC is a New Security Handshake Standard: An Overview of Security Analysis. *Computer* **2024**, *57*, 101–110.
15. Shor, P. Algorithms for quantum computation: discrete logarithms and factoring. In Proceedings of the Proceedings 35th Annual Symposium on Foundations of Computer Science. IEEE, 1994, pp. 124–134.
16. NIST. Post-Quantum Cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography. Accessed on 26 March 2026.
17. Migration to Post-Quantum Cryptography. NIST Crypto Agility Guide. Accessed on 14 February 2026.
18. ANSSI views on the Post-Quantum Cryptography transition. https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition. Accessed on 14 February 2026.
19. IETF Internet-Draft. Terminology for Post-Quantum Traditional Hybrid Schemes. https://www.ietf.org/archive/id/draft-ietf-pquip-pqt-hybrid-terminology-04.html, 2024. Accessed on 10 February 2026.
20. Bindel, N.; Brendel, J.; Fischlin, M.; Goncalves, B.; Stebila, D. Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange. In Proceedings of the PQCrypto 2019. Springer, 2019, pp. 206–226.
21. Barbosa, M.; Connolly, D.; Duarte, J.D.; Kaiser, A.; Schwabe, P.; Varner, K.; Westerbaan, B. X-Wing: The Hybrid KEM You've Been Looking For. *IACR Communications in Cryptology* **2024**, *1*. https://doi.org/10.62056/a3qj89n4e.
22. IETF Internet-Draft. X-Wing: General-Purpose Hybrid Post-Quantum KEM. https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/, 2025. Accessed on 5 March 2026.
23. IETF RFC 2865. Remote Authentication Dial In User Service (RADIUS). https://www.rfc-editor.org/rfc/rfc2865.html, 2000. Accessed on 5 March 2026.
24. IETF RFC 3579. RADIUS (Remote Authentication Dial In User Service) Support For Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3579.html, 2003. Accessed on 5 March 2026.
25. Fraile, L.P.; Tasopoulos, G.; Koulamas, C.; Zhao, R.K.; Sultan, N.H.; Regazzoni, F.; Fournaris, A.P. Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation. *IEEE Access* **2025**, *13*, 75861–75884.
26. Fraile, L.P.; Koulamas, C.; Fournaris, A.P. Reinventing EDHOC for the Post-Quantum Era. *IEEE Access* **2025**, *13*, 196622–196639. https://doi.org/10.1109/ACCESS.2025.3633843.
27. IETF Internet-Draft. KEM-based Authentication for EDHOC. https://www.ietf.org/archive/id/draft-pocero-authkem-edhoc-02.html, 2026. Accessed on 26 March 2026.
28. Shin, S.; Iskandar, A.P.S.; Ko, Y.; You, I. A Hybrid EDHOC Protocol. In Proceedings of the The 9th International Conference on Mobile Internet Security (MobiSec 2025), 2025.

29. Blanchet, B. An Efficient Cryptographic Protocol Verifier Based on Prolog Rules. In Proceedings of the CSFW, 2001.
30. Blanchet, B. Automated Verification of Correspondences for Security Protocols. *Journal of Computer Security* **2016**.
31. Lee, S. Open5GS: Open Source implementation of 5G Core and EPC. https://open5gs.org/, 2024. Accessed on 5 March 2026.
32. Güngör, A. UERANSIM: Open source 5G UE and RAN (gNodeB) simulator. https://github.com/aligungr/UERANSIM, 2024. Accessed on 5 March 2026.
33. The FreeRADIUS Server Project. FreeRADIUS: The world's most popular RADIUS Server. https://www.freeradius.org/, 2024. Accessed on 5 March 2026.
34. Iskandar, A.P.S. EdhocSecondaryAuth: Hybrid Post-Quantum EAP-EDHOC Secondary Authentication over Open5GS, UERANSIM, and FreeRADIUS (Source Code, ProVerif Models, and Benchmark Data). https://github.com/adipanca/EdhocSecondaryAuth, 2026. Accessed on 13 October 2026.
35. Jacomme, C.; Klein, E.; Kremer, S.; Racouchot, M. A comprehensive, formal and automated analysis of the EDHOC protocol. In Proceedings of the USENIX Security Symposium 2023. USENIX Association, 2023, pp. 5881–5898.
36. Norrman, K.; Sundararajan, V.; Bruni, A. Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices. In Proceedings of the SECRYPT 2021. SCITEPRESS, 2021, pp. 210–221.
37. Kim, J.; Duguma, D.G.; Lee, S.; Kim, B.; Lim, J.; You, I. Scrutinizing the Vulnerability of Ephemeral Diffie–Hellman over COSE (EDHOC) for IoT Environment Using Formal Approaches. *Mobile Information Systems* **2021**, *2021*.
38. Günther, F.; Mukendi, M.I.T. Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol. In Proceedings of the EuroS&P 2023. IEEE, 2023, pp. 773–796.
39. Cremers, C.; Dax, A.; Medinger, N. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In Proceedings of the CCS 2024. ACM, 2024, pp. 1046–1060.
40. Cottier, B.; Pointcheval, D. Security Analysis of Improved EDHOC Protocol. In Proceedings of the Foundations and Practice of Security (FPS) 2022. Springer, 2022, pp. 3–18.
41. Ferreira, L. Computational Security Analysis of the Full EDHOC Protocol. In Proceedings of the CT-RSA 2024. Springer, 2024, pp. 25–48.
42. Shin, S.; et al. A Hybrid EDHOC Protocol Post-Quantum Secure against Key Compromise Impersonation. In Proceedings of the Proceedings of eBISION 2026, 2026.
43. IETF RFC 10024. Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3. https://www.rfc-editor.org/rfc/rfc10024.html, 2026. Accessed on 29 October 2026.
44. IETF Internet-Draft. Use of ML-DSA in TLS 1.3 (draft-ietf-tls-mldsa-05). https://www.ietf.org/archive/id/draft-ietf-tls-mldsa-05.html, 2026. Accessed on 29 October 2026.
45. IETF RFC 5295. Specification for the Derivation of Root Keys from an Extended Master Session Key (EMSK). https://www.rfc-editor.org/rfc/rfc5295.html, 2008. Accessed on 5 March 2026.
46. IETF RFC 8613. Object Security for Constrained RESTful Environments (OSCORE). https://www.rfc-editor.org/rfc/rfc8613.html, 2019. Accessed on 5 March 2026.
47. NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final, 2024. Accessed on 2 February 2026.
48. CRYSTALS. https://pq-crystals.org/. Accessed on 10 February 2026.
49. ETSI TS 103 744. CYBER; Quantum-safe Hybrid Key Exchanges. https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.01.01_60/ts_103744v010101p.pdf, 2020. Accessed on 12 February 2026.
50. Angel, Y.; Dowling, B.; Hülsing, A.; Schwabe, P.; Weber, F. Post Quantum Noise. In Proceedings of the CCS 2022. ACM, 2022, pp. 97–109.
51. IETF RFC 9954. Hybrid Key Exchange in TLS 1.3. https://www.rfc-editor.org/rfc/rfc9954.html, 2026. Accessed on 29 October 2026.
52. IETF RFC 9370. Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc9370.html, 2023. Accessed on 11 February 2026.
53. IETF Internet-Draft. Composite ML-KEM for Use in the Internet X.509 Public Key Infrastructure and CMS. https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-kem-04.html, 2024. Accessed on 22 February 2026.
54. The PQXDH Key Agreement Protocol. https://signal.org/docs/specifications/pqxdh/. Accessed on 10 February 2026.
55. Fiedler, R.; Günther, F. Security Analysis of Signal's PQXDH Handshake. https://eprint.iacr.org/2024/702, 2024. Cryptology ePrint Archive: Paper 2024/702.
56. Dolev, D.; Yao, A.C. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* **1983**, *29*, 198–208. https://doi.org/10.1109/TIT.1983.1056650.
57. Krawczyk, H. SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols. In Proceedings of the CRYPTO 2003. Springer, 2003, pp. 400–425.

58. NIST FIPS 204. Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final, 2024. Accessed on 13 February 2026.
59. Dolev, D.; Yao, A. On the Security of Public Key Protocols. In Proceedings of the IEEE Symposium on Foundations of Computer Science, 1983.
60. Cremers, C.; Horvat, M.; Hoyland, J. A Symbolic Analysis of TLS 1.3. In Proceedings of the IEEE Symposium on Security and Privacy, 2017.
61. ISO/IEC 29128: Information technology – Security techniques – Verification of cryptographic protocols. International Organization for Standardization (ISO), ISO/IEC 29128, 2011.
62. The Libsodium Authors. Libsodium: A modern and easy-to-use crypto library. https://libsodium.org, 2024. Accessed: 2026-04-03.
63. Mbed TLS Contributors. Mbed TLS: Lightweight, modular, and portable cryptographic library. https://github.com/Mbed-TLS/mbedtls, 2024. Accessed: 2026-04-03.
64. PQClean Project Contributors. PQClean: Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean, 2024. Accessed: 2026-04-03.
65. Open Quantum Safe Project. liboqs: C library for prototyping and experimenting with quantum-resistant cryptography. https://github.com/open-quantum-safe/liboqs, 2024. Accessed on 5 March 2026.
66. Open Quantum Safe Project. oqs-provider: Open Quantum Safe Provider for OpenSSL 3. https://github.com/open-quantum-safe/oqs-provider, 2024. Accessed on 17 March 2026.
67. IETF RFC 9881. Internet X.509 Public Key Infrastructure: Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA). https://www.rfc-editor.org/rfc/rfc9881.html, 2025. Accessed on 29 October 2026.
68. 3GPP TS 23.502. Procedures for the 5G System (5GS). https://www.3gpp.org/DynaReport/23502.htm, 2024. Release 18. Accessed on 30 October 2026.
69. 3GPP TS 24.501. Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3. https://www.3gpp.org/DynaReport/24501.htm, 2024. Release 18. Accessed on 30 October 2026.
70. IETF RFC 5749. Distribution of EAP-Based Keys for Handover and Re-Authentication. https://www.rfc-editor.org/rfc/rfc5749.html, 2010. Accessed on 5 March 2026.
71. IETF RFC 6696. EAP Extensions for the EAP Re-authentication Protocol (ERP). https://www.rfc-editor.org/rfc/rfc6696.html, 2012. Accessed on 15 August 2026.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.